OBJECT ORIENTED SOFTWARE ENGINEERING

Prof. M.VENGATESHWARAN B.E., M.E., (Ph.D)
Assistant Professor (Grade-II)/CSE
Department of Computer Science and Engineering
R.M.K. Engineering College (Autonomous),
Chennai
E-mail:mkvengatesh@gmail.com
Website: www.vengatesh.xtgem.com

22CS602	OBJECT ORIENTED SOFTWARE ENGINEERING (Lab Integrated)	L	T	P	C
		2	0	2	3

OBJECTIVES:
The Course will enable learners to:

- Explain software engineering principles and activities involved in building large software programs.
- Describe the process of requirements gathering, analysis and unified modelling
- Illustrate the object oriented design process.
- Analyse various traditional and object oriented testing methods
- Apply estimation techniques, schedule project activities and compute pricing.

UNIT I	PRODUCT AND PROCESS	6+6

The Nature of Software – Defining the Discipline – The Software Process – Process models – Prescriptive Process Models – Product and Process – Agility and Process – What is an Agile Process? - Scrum – Other Agile Frameworks – Kanban – DevOps

List of Exercise/Experiments:

1. Identify a software system that needs to be developed.

2. Document the Software Requirements Specification (SRS) for the identified system.

UNIT II	REQUIREMENTS AND UNIFIED MODELING	6+6

Requirements Engineering – Establishing the Groundwork: Nonfunctional Requirements – Requirements Gathering – Developing Use Cases – Negotiating and Validating Requirements.
Unified Modeling Language – Introduction – Static and Dynamic Models – Modelling – Introduction to the UML – UML Diagrams – UML Class Diagrams – Use-Case Diagram – UML Dynamic Modelling.

List of Exercise/Experiments:
1. Identify use cases and develop the Use Case model.

2. Identify the conceptual classes and develop a Domain Model and also derive a Class Diagram from that.

UNIT III	OBJECT ORIENTED ANALYSIS AND DESIGN	6+6

Object oriented Analysis process – Business object Analysis – Use-case driven OOA – Business process modelling – Use case model. Design Concepts – Design Process – Design Concepts – Design Model: Design Principles and Design Elements. Architectural Design – Designing class-based components - Conducting Component Level Design – User Interface Analysis and Design – Pattern-Based Software Design.

List of Exercise/Experiments:
1. Using the identified scenarios, find the interaction between objects and represent them using UML Sequence and Collaboration Diagrams

2. Draw relevant State Chart and Activity Diagrams for the same system

UNIT IV	SOFTWARE TESTING	6+6

Component Level: A Strategic Approach to Software Testing – White-Box Testing – Black Box Testing - Object Oriented Testing Integration Level: Integration Testing – AI and Regression Testing – Integration Testing in the OO Context Specialized Testing for Mobility: Web application Testing – Web Testing Strategies – Security Testing – Performance Testing – Real time Testing – Testing AI Systems – Testing Virtual Environments.

List of Exercise/Experiments:

1. Implement the system as per the detailed design
2. Test the software system for all the scenarios identified as per the usecase diagram

UNIT V	SOFTWARE PROJECT MANAGAMENT	6+6

Software Metrics and Analytics: Software Measurement – Product Metrics. Creating a Viable Software Plan: The Project Planning Process – Software Scope and Feasibility – Decomposition and Estimation Techniques – Project Scheduling. Risk Management: Reactive Versus Proactive Risk Strategies – Risk Identification – Risk Projection – The RMMM Plan. Software Process Improvement: The SPI Process – The CMMI

List of Exercise/Experiments:

1. Improve the reusability and maintainability of the software system by applying appropriate design patterns.
2. Implement the modified system and test it for various scenarios

TOTAL: 30 + 30= 60 PERIODS

OUTCOMES:

Upon completion of the course, the students will be able to:

CO1: Summarize software engineering principles and activities involved in building large software programs

CO2: Describe the different phases of software development.

CO3: Explain the basics of OOAD and develop software using object oriented design.

CO4: Illustrate the different stages of the design process with a case study.

CO5: Develop miniprojects using the application of object oriented analysis and design.

CO6: Apply different testing strategies to develop efficient projects.

TEXT BOOKS:

1.Roger S. Pressman, “Software Engineering: A Practitioner‘s Approach” , McGraw Hill International Edition, Nineth Edition, 2020.

2.Ali Bahrami, “Object Oriented Systems Development”, McGraw Hill International Edition,2017.

REFERENCES:

1. Micheal Blalh and James Rumbaugh, Object Oriented Modeling and Design with UML, 2nd edition Pearson 2013.
2. Ian Sommerville, “Software Engineering”, Tenth Edition, Pearson Education, 2016.
3. Ivar Jacobson, Harold Bud Lawson, Pan-Wei Ng, Paul E. McMahon, Michael Goedicke, “The Essentials of Modern Software Engineering”, Morgan & Claypool Publishers, 2019.
4. Booch, G, Jacobson I, Rumbaugh J, “The Unified Modeling Language User Guide”, Addison Wesley, 2008.
5. Martin Fowler, “UML Distilled: A Brief Guide to the Standard Object Modeling Language”, 3rd edition, Addison Wesley, 2003.

1.1 Introduction to Software Engineering

AU : Dec.-13, 17, Marks 4

*"Software engineering is a **discipline** in which **theories**, **methods** and **tools** are applied to develop professional software **product**."*

In software engineering the **systematic** and **organized approach** is adopted. Based on the nature of the problem and development constraints various **tools and techniques** are applied in order to develop **quality software**.

The definition of software engineering is based on two terms :

- **Discipline :** For finding the solution to the problem an Engineer applies appropriate theories, methods and tools. While finding the solutions, Engineers must think of the organizational and financial constraints. Within these constraints only, he/she has to find the solution.
- **Product :** The software product gets developed after following systematic theories, methods and tools along with the appropriate management activities.

1.1.1 Defining Software

Software is nothing but a collection of computer programs and related documents that are intended to provide desired features, functionalities and better performance.

Software products may be :

1. **Generic** - That means developed to be sold to a range of different customers.
2. **Custom** - That means developed for a single customer according to their specification.

1.1.2 Software Characteristics

Software development is a logical activity and therefore it is important to understand basic characteristics of software. Some important characteristics of software are :

- **Software is engineered, not manufactured**
  - Software development and hardware development are two different activities.
  - A good design is a backbone for both the activities.
  - Quality problems that occur in hardware manufacturing phase can not be removed easily. On the other hand, during software development process such problems can be rectified.
  - In both the activities, developers are responsible for producing qualitative product.

- **Software does not ware out**
  - In early stage of hardware development process the failure rate is very high because of manufacturing defects. But after correcting such defects the failure rate gets reduced.
  - The failure rate remains constant for some period of time and again it starts increasing because of environmental maladies (extreme temperature, dusts, and vibrations).
  - On the other hand software does not get affected from such environmental maladies. Hence ideally it should have an *"idealized curve"*. But due to some **undiscovered errors** the failure rate is high and drops down as soon as the errors get corrected.
  - Hence in failure rating of software the *"actual curve"* is as shown below :

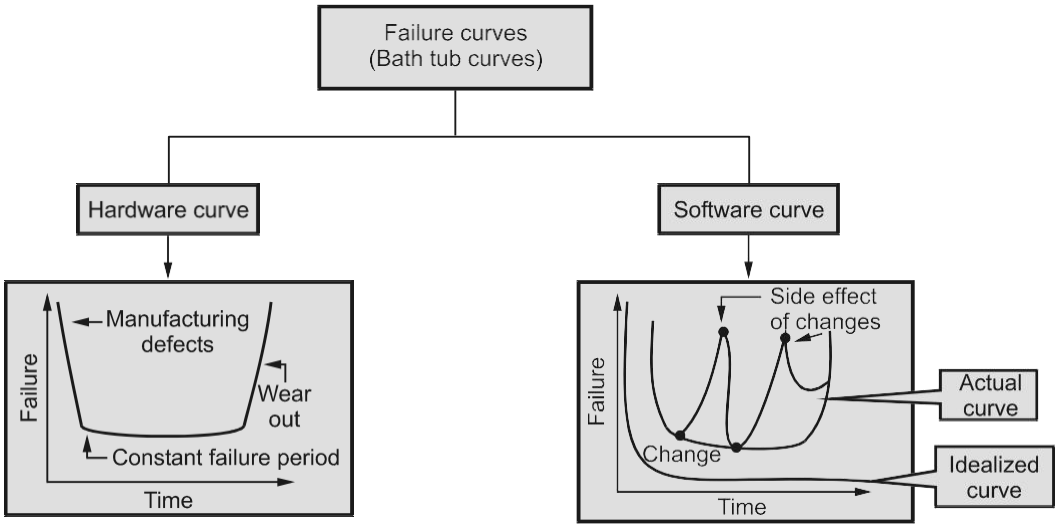


Fig. 1.1.1 Failure curves for hardware and software

  - During the life of software if any change is made, some **defects** may get **introduced**.
  - This causes failure rate to be high. Before the curve can return to original steady state another change is requested and again the failure rate becomes high.
  - Thus the failure curve looks like a spike. Thus frequent changes in software cause it to deteriorate.
  - Another issue with software is that there are ***no spare parts for software***.
  - If hardware component wears out it can be replaced by another component but it is not possible in case of software.
  - Therefore **software maintenance** is **more difficult** than the hardware maintenance.

- **Most software is custom built rather than being assembled from components**
  - While developing any hardware product firstly the circuit design with desired functioning properties is created.
  - Then required hardware components such as ICs, capacitors and registers are assembled according to the design, but this is not done while developing software product.
  - Most of the software is custom built.
  - However, now the software development approach is getting changed and we look for reusability of software components.
  - It is practiced to reuse algorithms and data structures.
  - Today software industry is trying to make library of reusable components.
  - **For example :** In today's software, GUI is built using the reusable components such as message windows, pull down menus and many more such components. The approach is getting developed to use in-built components in the software. This stream of software is popularly known as *component engineering.*

1.1.3 Categories of Software

Software can be applied in a situation for which a predefined set of procedural steps (algorithm) exists. Based on a complex growth of software it can be classified into following categories.

- **System software** - It is collection of programs written to service other programs. Typical programs in this category are compiler, editors, and assemblers. The purpose of the system software is to establish a communication with the hardware.
- **Application software** - It consists of standalone programs that are developed for specific business need. This software may be supported by database systems.
- **Engineering/Scientific software** - This software category has a wide range of programs from astronomy to volcanology, from automative stress analysis to space shuttle orbital dynamics and from molecular biology to automated manufacturing. This software is based on complex numeric computations.
- **Embedded software** - This category consists of program that can reside within a product or system. Such software can be used to implement and control features and functions for the end-user and for the system itself.
- **Web applications** - Web application software consists of various web pages that can be retrieved by a browser. The web pages can be developed using programming languages like JAVA, PERL, CGI, HTML, DHTML.

- **Artificial Intelligence software** - This kind of software is based on knowledge based expert systems. Typically, this software is useful in robotics, expert systems, image and voice recognition, artificial neural networks, theorem proving and game playing.

Review Questions

1. *What is the impact of reusability in software development process?* **AU : Dec.-17, Marks 4**
2. *Write a note on the unique characters of a software.* **AU : Dec.-17, Marks 3**

1.2 Goals and Objectives of Software

While developing software following are common objectives.

1. **Satisfy users requirements** - Many programmers simply don't do what the end user wants because they do not understand user requirements. Hence it becomes necessary to understand the demand of end user and accordingly software should be developed.
2. **High reliability** - Mistakes or bugs in a program can be expensive in terms of human lives, money, and customer relation. For instance, Microsoft has faced many problems because earlier release of windows has many problems. Thus software should be delivered only if high reliability is achieved.
3. **Low maintenance costs** - Maintenance of software is an activity that can be done only after delivering the software to the customer. Any small change in software should not cause restructuring of whole software. This indicates that the design of software has poor quality.
4. **Delivery on time** - It is very difficult to predict the exact time on which the software can be completed. But a systematic development of software can lead to meet the given deadline.
5. **Low production costs** - The software product should be cost effective.
6. **High performance** - The high performance software are expected to achieve optimization in speed and memory usage.
7. **Ease of reuse** - Use same software in different systems and software.

Environments reduce development costs and also improve the reliability. Hence reusability of developed software is an important property.

1.3 Difference between Software Product and Program

Sr. No.	Program	Software product
1.	Programs are developed by individual user and it is used for personal use.	Software product is developed by multiple users and it is used by large number of people or customers.
2.	Programs are small in size and possessing limited functionality.	Software product consists of multiple program codes, related documents such as SRS, design documents user manuals, test cases and so on.
3.	Generally only one person uses the program, hence there is a lack of user interface.	Good graphical user interface is most required by any software product.
4.	Program is generally developed by **programmer**.	Software product is developed by **software engineers** who are large in number and work in a team. Therefore systematic approach of developing software product must be applied.
5.	**For example :** Program of sorting n elements.	**For example :** A word processing software.

1.4 Layered Technology

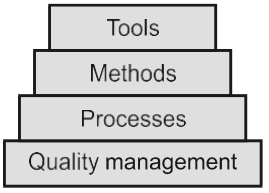


Fig. 1.4.1 Layered technology

- Software engineering is a layered technology. Any software can be developed using these layered approaches. Various layers on which the technology is based are **quality focus layer, process layer, methods layer, tools layer**.
- A disciplined **quality management** is a backbone of software engineering technology.
- **Process layer** is a foundation of software engineering. Basically, process defines the framework for timely delivery of software.
- In **method layer** the actual method of implementation is carried out with the help of requirement analysis, designing, coding using desired programming constructs and testing.
- Software **tools** are used to bring automation in software development process.
- Thus software engineering is a **combination** of process, methods, and tools for development of quality software.

1.5 Software Process

AU : Dec.-10,17, Marks 7

Software process can be defined as the structured set of activities that are required to develop the software system.

The fundamental activities are :

- Specification
- Validation
- Design and implementation
- Evolution

A software process model is an abstract representation of a process. It presents a description of a process from some particular perspective.

1.5.1 Common Process Framework

The process framework is required for representing the common process activities.

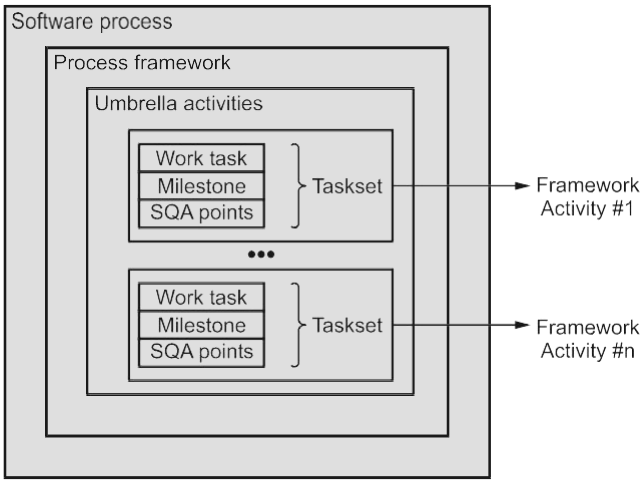


Fig. 1.5.1 Software process framework

As shown in Fig. 1.5.1, the software process is characterized by process framework activities, task sets and umbrella activities.

Process framework activities

- Communication
  - By communicating customer requirement gathering is done.
- Planning - Establishes engineering work plan, describes technical risks, lists resource requirements, work products produced and defines work schedule.
- Modeling - The software model is prepared by :
  - Analysis of requirements
  - Design

- Construction - The software design is mapped into a code by :
  - Code generation
  - Testing
- Deployment - The software delivered for customer evaluation and feedback is obtained.

Task sets - The task set defines the actual work done in order to achieve the software objective. The task set is used to adopt the framework activities and project team requirements using :

- Collection of software engineering work tasks
- Project milestones
- Software quality assurance points

Umbrella activities - The umbrella activities occur **throughout the process**. They focus on project management, tracking and control. The umbrella activities are

1. **Software project tracking and control** - This is an activity in which software team can **assess progress** and take corrective action to **maintain schedule**.
2. **Risk management** - The **risks** that may affect project outcomes or quality can be **analyzed**.
3. **Software quality assurance** - These are activities required to **maintain** software **quality**.
4. **Formal technical reviews** - It is required to **assess engineering work products to** uncover and **remove errors** before they propagate to next activity.
5. **Software configuration management** - Managing of configuration process when any **change** in the software occurs.
6. **Work product preparation and production -** The activities to create models, documents, logs, forms and lists are carried out.
7. **Reusability management -** It defines criteria for work product reuse.
8. **Measurement** - In this activity, the process can be defined and collected. Also **project and product measures** are used to assist the software team in delivering the required software.

1.5.2 Capability Maturity Model (CMM)

- The Software Engineering Institute (SEI) has developed a comprehensive process meta-model emphasizing **process maturity**. It is predicated on a **set of system and software capabilities** that should be present when organizations reach different levels of process capability and maturity.

- The Capability Maturity Model (CMM) is **used** in assessing how well an organization's processes allow **to complete and manage** new software projects.
- Various process maturity levels are :

Level 1 : Initial - Few processes are defined and individual efforts are taken.

Level 2 : Repeatable - To track cost schedule and functionality basic project management processes are established. Depending on earlier successes of projects with similar applications necessary process discipline can be repeated.

Level 3 : Defined - The process is standardized, documented and followed. All the projects use documented and approved version of software process which is useful in developing and supporting software.

Level 4 : Managed - Both the software process and product are quantitatively understood and controlled using detailed measures.

Level 5 : Optimizing - Establish mechanisms to plan and implement change. Innovative ideas and technologies can be tested.

Thus CMM is used for improving the software project.

Review Question

1. *Explain the CMMI model to access the organization level.* **AU : Dec.-17, Marks 7**

1.6 Prescriptive Process Models

AU : May-05,06,09,15,16,17,18, Dec.-06,09,11,16,17,19, Marks 16

- **Definition of Process Model :** The process model can be defined as the **abstract representation of process**. The appropriate process model can be chosen based on abstract representation of process.
- The software process model is also known as **Software Development Life Cycle (SDLC) Model** or software paradigm.
- These models are called **prescriptive process models** because they are following some rules for correct usage.
- In this model various activities are carried out in some specific sequence to make the desired software product.
- Various prescriptive process models are -

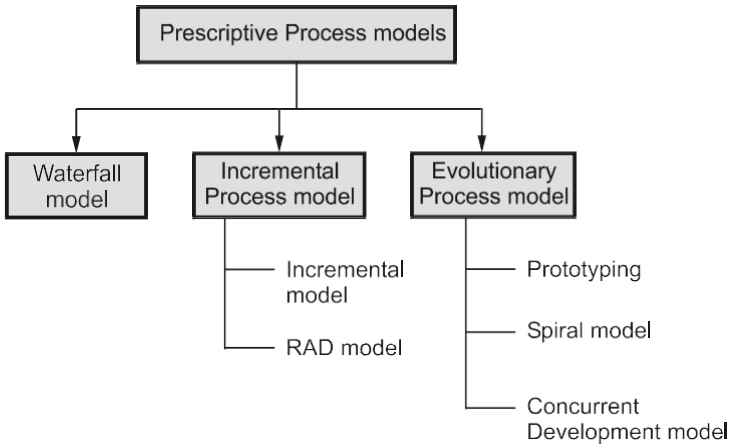


Fig. 1.6.1 Prescriptive process model

1.6.1 Need for Process Model

The software development team must decide the process model that is to be used for software product development and then the entire team must adhere to it. This is necessary because the software product development can then be done systematically. Each team member will **understand - what is the next activity** and **how** to do it. Thus **process model** will bring the **definiteness and discipline** in overall development process.

Every process model consists of **definite entry and exit criteria** for **each phase**. Hence the transition of the product through various phases is definite. If the process model is not followed for software development then any team member can perform any software development activity, this will ultimately cause a chaos and software project will definitely fail without using process model, it is difficult to monitor the progress of software product. Let us discuss various process models one by one.

1.6.2 Waterfall Model

- The waterfall model is also called as '**linear-sequential model**' or '**classic life cycle model**'. It is the oldest software paradigm. This model suggests a systematic, **sequential approach** to software development.
- The software development starts with **requirements gathering phase**. Then progresses through **analysis**, **design**, **coding**, **testing** and **maintenance**. Following figure illustrates waterfall model.
- In **requirement gathering and analysis** phase the **basic requirements** of the system must be understood by software engineer, who is also called **Analyst**. The **information domain**, **function**, **behavioural requirements** of the system are

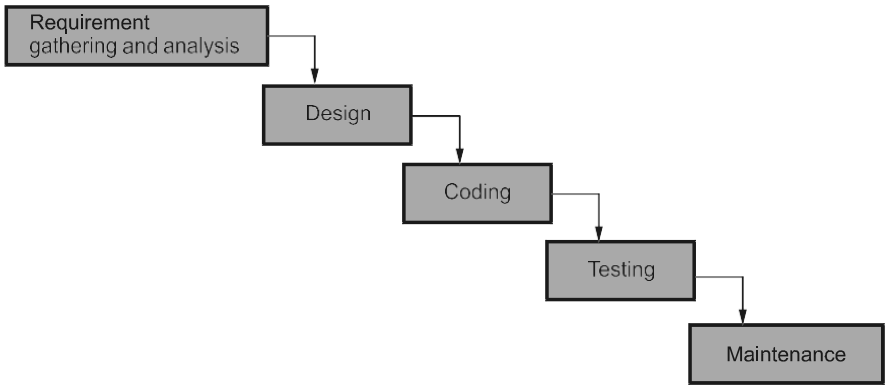


Fig. 1.6.2 Waterfall model

understood. All these requirements are then **well documented** and discussed further with the customer, for reviewing.

- The **design** is an intermediate **step between** requirements analysis and coding.

  Design focuses on program attributes such as -

  - Data structure
  - Software architecture
  - Interface representation
  - Algorithmic details.

  The requirements are translated in some easy to represent form using which coding can be done effectively and efficiently. The design needs to be documented for further use.

- **Coding** is a step in which design is translated into **machine-readable form**. If design is done in sufficient detail then coding can be done effectively. **Programs** are created in this phase.
- **Testing** begins when coding is done. While performing testing the major focus is on **logical internals of the software**. The testing ensures execution of all the paths, functional behaviours. The purpose of testing is to **uncover errors**, **fix the bugs** and **meet the customer requirements**.
- **Maintenance** is the **longest life cycle phase**. When the system is installed and put in practical use then error may get introduced, correcting such errors and **putting it in use** is the **major purpose** of maintenance activity. Similarly, **enhancing system's services** as new requirements are discovered is again maintenance of the system.

This model is widely used model, although it has many drawbacks. Let us discuss benefits and drawbacks.

Benefits of waterfall model

1. The waterfall model is **simple to implement**.
2. For implementation of **small systems** waterfall model is useful.

Drawbacks of waterfall model

There are some **problems** that are encountered if we apply the **waterfall model** and those are :

1. It is difficult to follow the sequential flow in software development process. If some changes are made at some phases then it may cause some confusion.
2. The requirement analysis is done initially and sometimes it is not possible to state all the requirements explicitly in the beginning. This causes difficulty in the project.
3. The customer can see the **working model** of the project only **at the end**. After reviewing of the working model; if the customer gets dissatisfied then it causes serious problems.
4. Linear nature of waterfall model induces **blocking states**, because certain tasks may be dependant on some previous tasks. Hence it is necessary to accomplish all the dependant tasks first. It may cause long waiting time.

Example 1.6.1 *Explain how water-fall model is applicable for the development of the following systems:*
a) University accounting system
b) Interactive system that allows railway passengers to find time and other information from the terminals installed in the station.

Solution :

a) **University accounting system :** If the software developers who have the experience in developing the account systems then building university account system based on existing design could be easily managed with water-fall model.

b) **Interactive system that allows railway passengers to find time and other information from the terminals installed in the station.**

For developing such interactive system, all the requirements must be correctly identified and analyzed before the designing of the project. The requirements of end-users must be correctly and un-ambiguously understood by the developers prior to design phase. Once the requirements are well defined then using disciplined design, coding and testing phases the required system can be built using water-fall model.

1.6.3 Incremental Process Model

In this model, the initial model with limited functionality is created for user's understanding about the software product and then this model is refined and expanded in later releases.

1.6.3.1 Incremental Model

- The incremental model has same phases that are in waterfall model. But it is iterative in nature. The incremental model has following phases.

  1. Analysis 2. Design

  3. Code 4. Test

- The incremental model delivers series of releases to the customer. These releases are called **increments**. More and more functionality is associated with each increment.

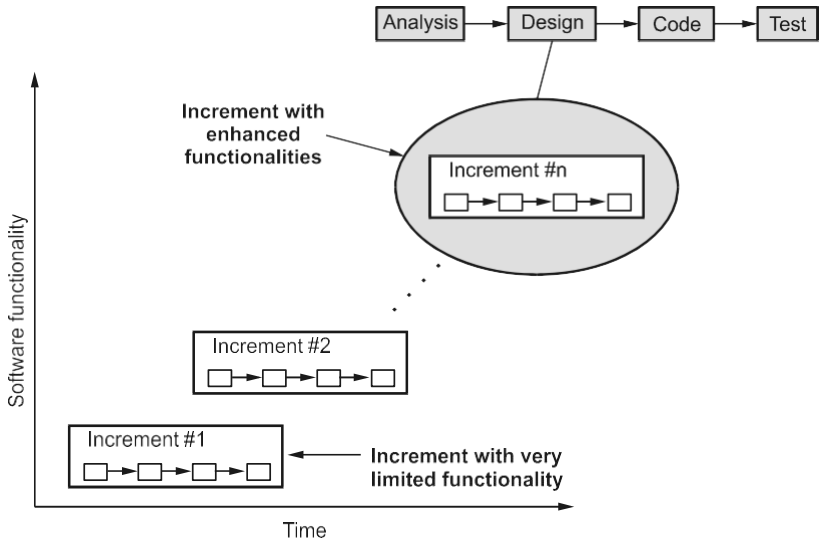


Fig. 1.6.3 The incremental model

- The first increment is called **core product**. In this release the basic requirements are implemented and then in subsequent increments new requirements are added.
- The word processing software package can be considered as an example of incremental model. In the first increment only the document processing facilities are available. In the second increment, more sophisticated document producing and processing facilities, file management functionalities are given. In the next increment spelling and grammar checking facilities can be given. Thus in incremental model progressive functionalities are obtained with each release.

When to choose it ?

1. When requirements are reasonably well-defined.
2. When overall scope of the development effort suggests a purely linear effort.
3. When limited set of software functionality needed quickly.

Merits of incremental model

1. The incremental model can be adopted when there are less number of people involved in the project.
2. Technical risks can be managed with each increment.
3. For a very small time span, atleast core product can be delivered to the customer.

1.6.3.2 RAD Model

- The RAD Model is a type of incremental process model in which there is **extremely short development cycle.**
- When the requirements are fully understood and the **component based construction** approach is adopted then the RAD model is used.
- Using the RAD model the fully functional system can be developed within **60 to 90 days.**
- Various phases in RAD are Requirements Gathering, Analysis and Planning, Design, Build or Construction and finally Deployment.
- Multiple teams work on developing the software system using RAD model parallely.
- In the **requirements gathering** phase the developers communicate with the users of the system and understand the business process and requirements of the software system.
- During **analysis and planning** phase**,** the analysis on the gathered requirements is made and a planning for various software development activities is done.
- During the **design** phase various models are created. Those models are Business model, data model and process model.
- The **build** is an activity in which using the existing software components and automatic code generation tool the implementation code is created for the software system. This code is well tested by its team. The functionalities developed by all the teams are integrated to form a whole.
- Finally the deployment of all the software components (created by various teams working on the project) is carried out.

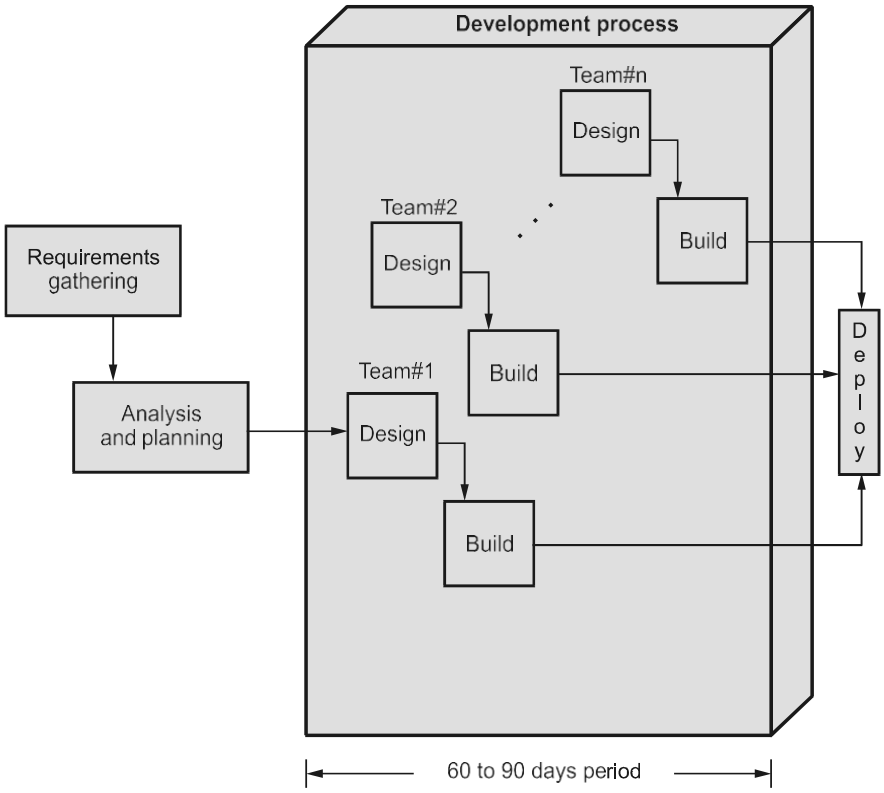


Fig. 1.6.4 Rapid application development

Drawbacks of rapid application development

1. It requires **multiple teams** or large number of people to work on the scalable projects.
2. This model requires **heavily committed developer** and customers. If commitment is lacking then RAD projects will fail.
3. The projects using RAD model requires **heavy resources**.
4. If there is no appropriate modularization then RAD projects fail. Performance can be problem to such projects.
5. The projects using RAD model find it difficult to adopt new technologies.

Example 1.6.2 *Which type of applications suit RAD model ? Justify your answer*

Solution : The RAD model is suitable for information system applications, business applications and the for systems that can be modularized because of following reasons -

1. This model is similar to waterfall model but it uses very short development cycle.
2. It uses component-based construction and emphasises reuse and code generation.

3. This model uses multiple teams on scaleable projects.
4. The RAD model is suitable for the projects where technical risks are not high.
5. The RAD model requires heavy resources.

Example 1.6.3 *Provide three examples of software projects that would be amenable to incremental model. Be specific*

Solution : There can various examples of software projects that would be amenable to incremental model. For instance -

1. **Banking software service :** This service can be personal service. That means for personal banking system the incremental model can be used. In later state of increments, this system can implement insurance service, home loans and some other features of banking services.
2. **Web browser application :** The base application can be developed and distributed. This is the basic increment of the application. In the later increments the plugins can be provided to enhance the experience of web browser applications.
3. **Operating system software :** The operating system software providing the basic system handling functionalities is the first increment. After the release of the basic versions then updates or security patches are provided to the customer in the form of increments. Various distribution package in the form of versions such as basic home edition, premium, ultimate and so on can be the increments of operating system software.

1.6.4 Evolutionary Process Model

While developing the software systems, it is often needed to make modifications in earlier development phases or the tasks sets. If the development process is linear or in a straight line (from requirements gathering to deployment) then the end product will be unrealistic. In such cases, the **iterative approach** needs to be adopted. The evolutionary process model is **iterative model**.

1.6.4.1 Prototyping

- In prototyping model initially the requirement gathering is done.
- Developer and customer define overall objectives; identify areas needing more requirement gathering.
- Then a quick design is prepared. This design represents what will be visible to user in input and output format.
- From the quick design a prototype is prepared. Customer or user evaluates the prototype in order to refine the requirements. Iteratively prototype is tuned for satisfying customer requirements. Thus prototype is important to identify the software requirements.

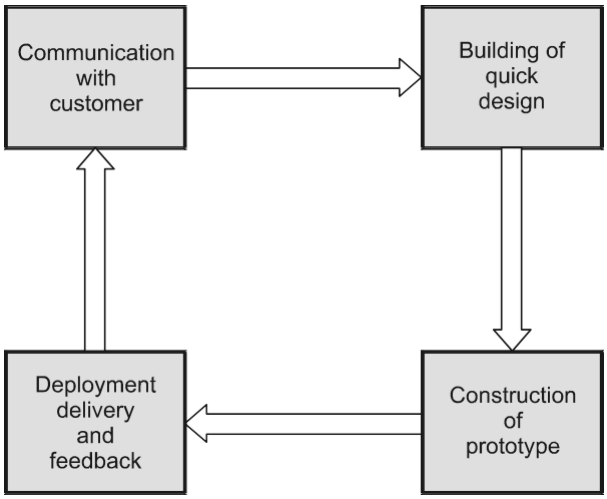


Fig. 1.6.5 Prototyping

- When working prototype is built, developer use existing program fragments or program generators to throw away the prototype and rebuild the system to high quality.
- Certain classes of mathematical algorithms, subset of command driven systems and other applications where results can be easily examined without real time interaction can be developed using prototyping paradigm.

When to choose it ?

- Software applications that are relatively easy to prototype almost always involve Human-machine Interaction (HCI) the prototyping model is suggested.
- A general objective of software is defined but not detailed input, processing or output requirements. Then in such a case prototyping model is useful.
- When the developer is unsure of the efficiency of an algorithm or the adaptability of an operating system then prototype serves as a better choice.

Drawbacks of prototyping

1. In the first version itself, customer often wants "few fixes" rather than rebuilding of the system whereas rebuilding of new system maintains high level of quality.
2. The first version may have some compromises.
3. Sometimes developer may make implementation compromises to get prototype working quickly. Later on developer may become comfortable with compromises and forget why they are inappropriate.

Comparison between prototyping and incremental process model

Sr. No.	Prototyping	Incremental process model
1.	Some requirements are gathered initially, but there may be change in requirements when the working prototype is shown to the customer.	The requirements are precisely defined and there is no confusion about the final product of the software.
2.	The development team has adequate domain knowledge. Similarly they can adopt the new technologies if product demands.	The development team with less domain knowledge can be accommodated due to iterative nature of this model. The change in technology in the later phase can not be tolerated.
3.	All the end-users are involved in all phases of development.	All the end-users need not be involved in all the phases of development.
4.	There can be use of some reusable software components in project development process.	There is no use of reusable components in development process.

1.6.4.2 Spiral Model

- This model possess the **iterative nature** of prototyping model and controlled and **systematic approaches** of the linear sequential model.
- This model gives efficient development of incremental versions of software. In this model, the software is developed in series of increments.
- The sprial model is divided into a number of **framework activities**. These framework activities are denoted by **task regions**.
- Usually there are **six tasks regions**. The spiral model is as shown in Fig. 1.6.6. (See Fig. 1.6.6 on next page)
- Spiral model is realistic approach to development of **large-scale systems** and software. Because customer and developer better understand the problem statement **at each evolutionary level**. Also **risks** can be identified or rectified at each such level.
- In the **initial pass**, product specification is built and in subsequent passes around the spiral the prototype gets developed and then more improved versions of software gets developed.
- During **planning phase**, the cost and schedule of software can be planned and adjusted based on feedback obtained from customer evaluation.
- In spiral model, project **entry point axis** is defined. This axis represents starting point for different types of projects.
- For instance, concept development project will start at core of spiral and will continue along the spiral path. If the concept has to be developed into actual project then at entry point 2 the product development process starts. Hence entry

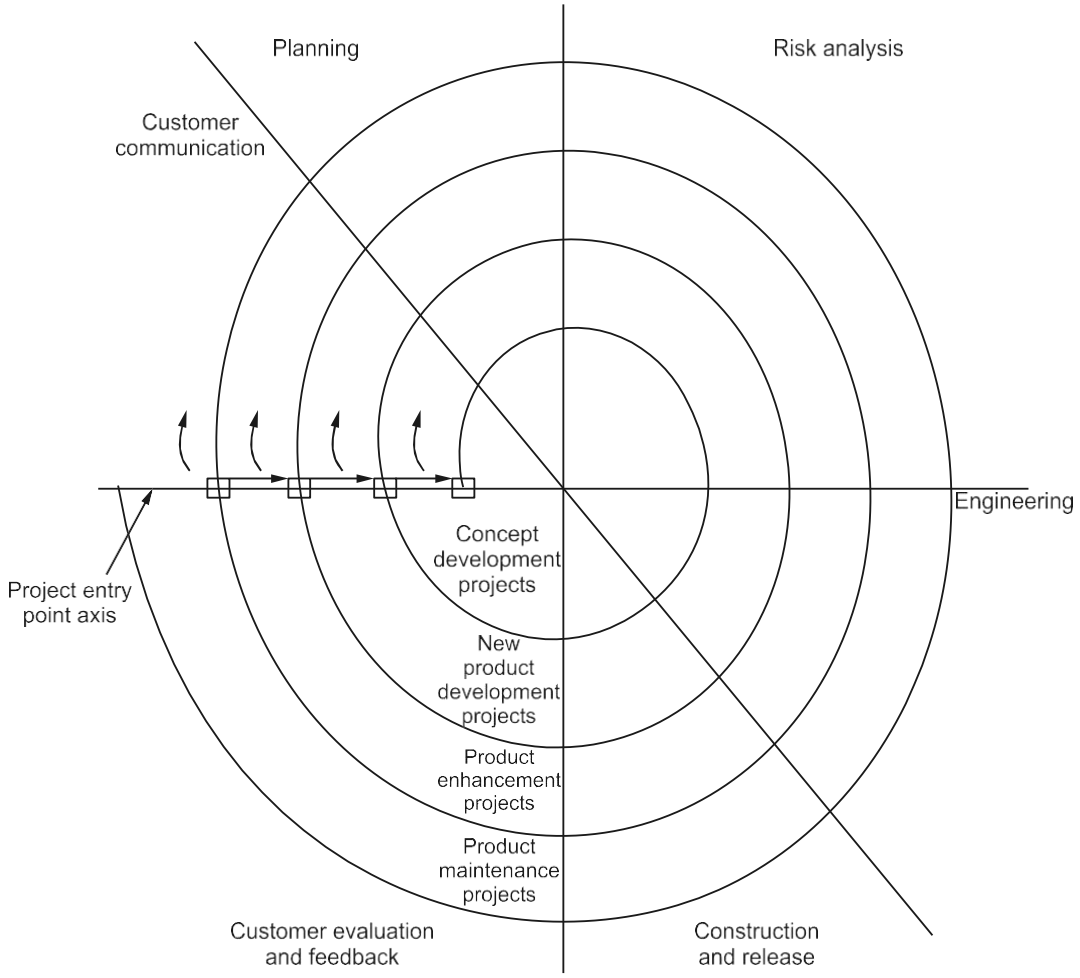


Fig. 1.6.6 Spiral model

point 2 is called product development project entry point. The development of the project can be carried out in iterations.

- The task regions can be described as :

  i) Customer communication - In this region, it is suggested to establish customer communication.

  ii) Planning - All planning activities are carried out in order to define resources time line and other project related activities.

  iii) Risk analysis - The tasks required to calculate technical and management risks are carried out.

  iv) Engineering - In this task region, tasks required to build one or more representations of applications are carried out.

v) **Construct and release -** All the necessary tasks required to construct, test, install the application are conducted. Some tasks that are required to provide user support are also carried out in this task region.

vi) **Customer evaluation -** Customer's feedback is obtained and based on customer evaluation required tasks are performed and implemented at installation stage.

- In each region, number of **work tasks** are carried out depending upon the characteristics of project. For a small project relatively small number of work tasks are adopted but for a complex project large number of work tasks can be carried out.
- In spiral model, the software engineering team **moves around the spiral** in a clockwise direction beginning at the core.

Advantages of spiral model

- Requirement changes can be made at every stage.
- **Risks can be identified** and rectified before they get problematic.

Drawbacks of spiral model

- It is based on **customer communication**. If the communication is not proper then the software product that gets developed will not be up to the mark.
- It demands considerable **risk assessment**. If the risk assessment is done properly then only the successful product can be obtained.

When to choose it ?

1. When the prototypes for the software functionality are needed.
2. When requirements are **not very clearly defined** or complex.
3. When the **large or high budget** projects need to be developed.
4. When the **risk assessment** is very critical and essential
5. When project is not expected within a specific limited time span.

Comparison between Spiral model and Prototyping model

Sr. No.	Spiral model	Prototyping model
1.	The development team with less domain knowledge can be accommodated due to iterative nature of this model. The change in technology in the later phase can not be tolerated.	The development team has adequate domain knowledge. Similarly they can adopt the new technologies if product demands.

2.	All the end-users need not be involved in all the phases of development.	All the end-users are involved in all phases of development.
3.	Funding are not stable for the projects that can be developed using spiral model.	Funding are stable for these type of projets.
4.	The requirements that are gathered and analyzed are high reliability requirements.	Some requirements are gathered initially, but there may be change in requirements when the working prototype is shown to the customer.

Difference between waterfall model and spiral model

Sr. No.	Waterfall model	Spiral model
1.	It requires well **understanding of requirements** and familiar technology.	It is developed in **iterations**. Hence the requirements can be identified at new iterations.
2.	**Difficult** to accommodate **changes** after the process has started.	The required changes can be made at every stage of **new version**.
3.	Can accommodate iteration but indirectly.	It is iterative model.
4.	**Risks** can be identified **at the end** which may cause failure to the product.	**Risks** can be identified and **reduced** before they get problematic.
5.	The customer can see the **working model** of the project only **at the end**. After reviewing of the working model; if the customer gets dissatisfied then it causes serious problems.	The customer can see the **working product at certain stages** of iterations.
6.	**Customers prefer** this model.	**Developers prefer** this model.
7.	This model is good for **small systems**.	This model is good for **large systems**.
8.	It has **sequential** nature.	It has **evolutionary** nature.

1.6.4.3 Concurrent Development Model

- The concurrent development model is also called as **concurrent engineering.**
- In this model, the framework activities or software development tasks are represented as **states.**
- The **modeling** or **designing** phase of software development can be in one of the states like *under development*, *waiting for modification*, *under revision* or *under review* and so on.
- Fig. 1.6.7 represents these states.

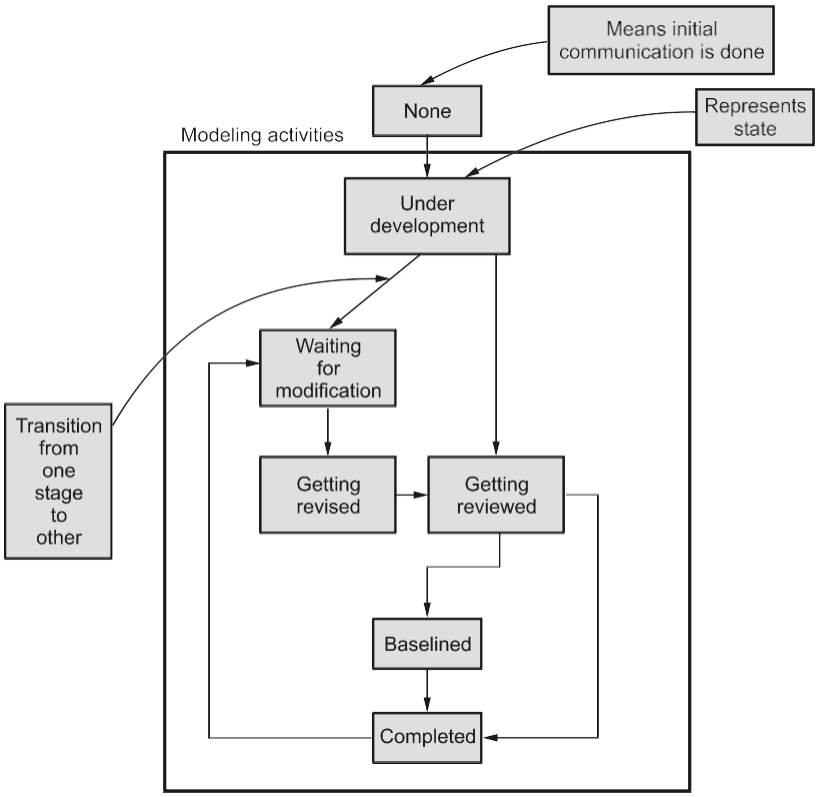


Fig. 1.6.7 Concurrent development model

- All the software development activities exist concurrently in this model but these activities can be in various **states.**
- These states make transitions. That is during **modeling,** the transition from **under development** state to **waiting for modification** state occurs.
- This model basically defines the **series of events** due to which the transition from one state to another state occurs. This is called **triggering.** These series of events occur for every software development activity, action or task.
- This model defines various activities that occur concurrently and a network of activities is defined.

Advantages

1. All types of software development can be done using concurrent development model.
2. This model provides accurate picture of current state of project.
3. Each activity or task can be carried out concurrently. Hence this model is an efficient process model.

Example 1.6.4 *Compare and contrast the different life cycle models.* **AU : Dec.-11, Marks 6**

Solution :

Waterfall model	Spiral model	Prototyping model	Incremental model
Requirements must be clearly understood and defined at the beginning only.	The **requirements** analysis and gathering can be done in iterations because requirements get changed quite often.	**Requirements analysis** can be made in the later stages of the development cycle, because requirements get changed quite often.	The **requirements analysis** can be made in the later stages of the development cycle.
The **development team** having the adequate experience of working on the similar project is chosen to work on this type of process model	The **development team** having less experience of working on the similar projects is allowed in this process model	The **development team** having less experience of working on the similar projects is allowed in this process model.	The **development team** having the adequate experience of working on the similar project is chosen to work on this type of process model.
There is **no user involvement** in all the phases of development process.	There is **no user involvement** in all the phases of development process.	There is **user involvement** in all the phases of development process.	There is **user involvement** in all the phases of development process.
When the requirements are reasonably well defined and the development effort suggests a purely linear effort then the **waterfall model** is **chosen**.	Due to iterative nature of this model, the risk identification and rectification is done before they get problematic. Hence for handling real time problems the **spiral model is chosen**.	When developer is unsure about the efficiency of an algorithm or the adaptability of an operating system then the **prototyping model is chosen.**	When the requirements are reasonably well defined, the development effort suggests a purely linear effort and when limited set of software functionality is needed quickly then the **incremental model is chosen.**

Example 1.6.5 *For the scenario described below, which life cycle model would you choose? Give the reason why you would choose this model.*

You are interacting with the MIS department of a very large oil company with multiple departments. They have a complex legacy system. Migrating the data from this legacy system is not an easy task and would take a considerable time. The oil company is very particular about processes, acceptance criteria and legal contracts. **AU : Dec.-11, Marks 5**

Solution : Legacy applications are database management systems. The software companies normally migrate these applications to some relational databases. For data migrating projects a cohesive method that enables the developer to cycle or spiral through

the migration process is required. In this migrating process following steps need to be followed -

1. Analyze the data.
2. Extract the data which is to be transformed.
3. Transform the extracted data.
4. Validate the transformed data.
5. Load the validated data to the target system.

These above mentioned steps are repeated until the migration is successfully completed. The risk management must be done properly. All these factors suggest that the spiral model is suitable for the given project.

Example 1.6.6 *Describe at least one scenario where*

1. RAD model would be applicable and not the waterfall model.

2. Waterfall model is preferable to all other models. **AU : Dec.-11, Marks 10**

Solution : 1. RAD model would be applicable and not the waterfall model :

Following are some scenario where RAD model would be applicable and not the waterfall model.

A) The projects in which users are involved in all phases.

B) The projects in which users are experts of problem domain.

C) The project is enhancement of existing system.

2. Waterfall model is preferable to all other models :

In the following scenario waterfall model is preferable to all other models.

The project for which requirements are easily understandable and defined.

Example 1.6.7 *How does a spiral model represent a process suitable to represent a real time problem ?* **AU : CSE, May-05, Marks 8**

Solution : Spiral model represents a process suitable to represent a real time problem because of following reasons -

1. Software evolves as the project progresses. And at every evolutionary level the risks are identified and managed and risks are reduced at every stage.
2. It enables the developer to apply the prototype approach at any stage in the evolution of the product. It helps in adopting the approach systematic stepwise development of the product.
3. The iterative frameworks help in analyzing the product at every evolutionary stage.

4. The spiral model demands a direct consideration of technical risks at all stages of project. The risks are reduced before they get problematic.

Example 1.6.8 *Which type of applications suit RAD model ? Justify your answer.*

AU : May-06, Marks 10

Solution : The RAD model is suitable for information system applications, business applications and the for systems that can be modularized because of following reasons -

1. This model is similar to waterfall model but it uses very short development cycle.
2. It uses component-based construction and emphasises reuse and code generation.
3. This model uses multiple teams on scaleable projects.
4. The RAD model is suitable for the projects where technical risks are not high.
5. The RAD model requires heavy resources.

Example 1.6.9 *Discuss the major differences between software life cycle model and a process model.*

AU : CSE, Dec.-06, Marks 4

Solution :

Software life cycle model	Process model
This model is based on common four activities : analysis, design, code and testing.	This model is based on problem definition, technical development, software integration and existing status.
The software development process can be clearly and systematically defined in phases.	The software development process cannot be clearly defined in phases.
Customer interaction is possible in every stage of software development process in this model.	Customer interaction is only at initial stage of software development.
Large scale projects can be handled using this approach.	Large scale projects may be caught in chaotic situation using this approach.

Example 1.6.10 *As you move outward along with process flow path of the spiral model, what can we say about the software that is being developed or maintained ?*

AU : CSE, May.-09, Marks 4

Solution : When software engineering team moves around the spiral, the first circuit around the spiral results in development of product specification. The subsequent passes around the spiral might be used to develop prototype in more subsequent manner. In each pass, through planning region, some adjustments to project plan are made. Cost and schedule adjustments can also be made according to customer feedback.

Example 1.6.11 *A software project which is considered to be very simple and the customer is in position of giving all the requirements at the initial stage, which process model would you prefer for developing the project ?* **AU : IT, Dec.-09, Marks 16**

Solution : The linear sequential model or a waterfall model is appropriate when a simple project has to be developed with already known requirements.

Waterfall model - Refer section 1.6.2

Example 1.6.12 *Assume that you are the technical manager of a software development organization. A client approached you for a software solution. The problems stated by the client have uncertainties which lead to loss if it not planned and solved. Which software development model you will suggest for this project – justify. Explain that model with its pros and cons and neat sketch.* **AU : Dec.-16, Marks 16**

Solution : The uncertainties in problem statement leads to risks in the project. Hence the spiral model can be used for development of such project.

Spiral model - Refer section 1.6.4.2.

Example 1.6.13 *What is the role of user participation in the selection of a life cycle model?* **AU : May-16, Marks 8**

Solution : Due to users' participation , their understanding about the project or system increases. Following are the issues that depict the role of user participation in selection of life cycle model.

1. Prototype model and RAD model are the life cycle model in which **user involvement** is expected in **all the phases.**
2. There is no user involvement expected in all the phases of waterfall, spiral model and evolutionary development model.
3. The waterfall model, spiral model, iterative enhancement model or evolutionary development model **do not require limited user participation.**
4. RAD model and Prototype model does not allow limited user participation.
5. Prototype model, RAD Model, Evolutionary development model require that the **users must be experts of problem domain.** They should understand the system properly well in advance.
6. For the selection of waterfall model, it is **not** necessary that the **user has expertise of problem** domain.
7. The prototype, iterative enhancement model, spiral model and Evolutionary model can be selected by the users **having no previous experience of participation in similar projects.**
8. For selection of waterfall model and prototype model, the users must have **participated earlier in similar type of projects.**

Example 1.6.14 *Describe the type of situations where iterative enhancement model might lead to difficulties.* **AU : May-16, Marks 8**

Solution :

1. In the prototyping model, during the first prototype itself, the customer often wants **few fixes** rather than rebuilding of new system.
2. Sometimes developer may make implementation compromises to get prototype working quickly. But later on the developer may become **comfortable with compromises** and forget why they were inappropriate.
3. The spiral model is mainly based on customer communication. If the **communication** is **not proper** then the software product that gets developed will not be up to the mark.
4. During iterative enhancement cycle of project development, if the **risk assessment** is **not done** properly then the product will become total failure.

Example 1.6.15 *What is process model ? Describe the process model that you would choose to manufacture a car. Explain by giving suitable reasons.* **AU : May-17, Marks 13**

Solution : Process model : Refer section 1.6.

The process model that can be used for car manufacture is **spiral model.**

Spiral model : Refer section 1.6.4.2.

Reasons :

1) This is a model in which **risks** can be **identified** and rectified before they get problematic.
2) The **iterative framework** of this model helps in analyzing the problem at every evolutionary stage.
3) The required **changes can be made at every stage** of new version.
4) This model is **good for large systems**.

Review Questions

1. *Neatly explain the following process models and write their advantages and disadvantages. i) Spiral model, ii) Rapid application development model.* **AU : May-15, Marks 8**
2. *Discuss the prototyping model. What is the effect of designing a prototype on the overall cost of the software project ?* **AU : May-16, Marks 8**
3. *Which process model is best suited for risk management ? Discuss in detail with an example. Give the advantages and disadvantages of the model.* **AU : Dec.-16, Marks 16**
4. *What is the significance of the spiral model when compared with other models.* **AU : Dec-17, Marks 3**

5. *Which software process model is good for risk management ? Explain the model. Describe how the model is used to layout the objectives, risks and plans for quality improvement.*

AU : May-18, Marks 16

6. *Outline the spiral life cycle model with a diagram.* **AU : Dec.-19, Marks 13**

Example 1.7.2 *Compare the following life cycle models based on their distinguishing factors, strengths and weaknesses, - Waterfall Model, RAD Model, Spiral Model and Formal Methods Model. (Present in the form of table only - use diagrams wherever necessary)*

AU : Dec.-13,14, Marks 16; May-19, Marks 13

OR

Elucidate the key features of the software process models with suitable examples.

AU : May-16, Marks 8

Solution :

Waterfall model	RAD model	Spiral model	Formal methods model
Requirements must be **clearly understood** and defined at the beginning only.	**Requirements** must be **clearly understood** and defined at the beginning only.	The requirements analysis and gathering can be done in iterations because **requirements get changed quite often**.	**Requirements** must be **clearly understood** and defined at the beginning only.
The **development team** having the adequate experience of working on the similar project is chosen to work on this type of process model.	The **development team** having the adequate experience of working on the similar project is chosen to work on this type of process model.	The **development team** having less experience of working on the similar projects is allowed in this process model.	The **development team** having the adequate experience of working on the similar project is chosen to work on this type of process model.
If the development team has **less domain knowledge** or if it new to the technology then such a team is allowed for this kind of process model.	If the development team has **less domain knowledge** or if it new to the technology then such a team is **not allowed** for this kind of process model.	If the development team has **less domain knowledge** or if it new to the technology then such a team is allowed for this kind of process model.	If the development team has **less domain knowledge** or if it new to the technology then such a team is **not allowed** for this kind of process model.

There is **no user involvement** in all the phases of development process.	There is **user involvement** in all the phases of development process.	There is **no user involvement** in all the phases of development process.	**Limited community** makes **use** of formal methods model for their projects because this methodology is based on mathematical theorems, formal methods and automata theory.
Types of Projects : When the **requirements** are reasonably **well defined** and the development effort suggests a purely linear effort then the waterfall model is chosen	When there is a **tight project schedule** or project is **enhancement** of the **existing projects** or if there is use of **reusable components** in the project then for developing such projects this process model is suitable.	Due to iterative nature of this model, the risk identification and rectification is done before they get problematic. Hence for handling real time problems the **spiral model is chosen.**	Whenever there is a need to build the **scientific models** based on mathematical techniques, or **simulation** of some **real time systems** or when there is a need to build the systems that can contribute to the reliability and robustness (normally **critical systems**) then the formal methods models are used.

1.8 Introduction to Agility

- The Agile Manifesto, also called the **Manifesto for Agile Software Development,** is a formal declaration of **four key values** and **12 principles** to guide an iterative and people-centric approach to software development.
- The agile methods were developed to overcome the weakness of conventional software engineering.
- The agile manifesto is represented by following Fig. 1.8.1.

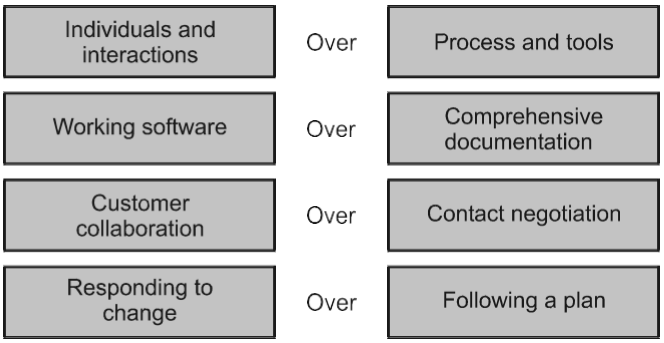


Fig. 1.8.1 Agile manifesto

1.9 Agile Process

AU : Dec.-16, 19, May-19, Marks 4

An Agile Process is a development process that:

- Breaks work into small iterations (sprints)
- Delivers working product increments frequently
- Encourages continuous feedback and improvement
- Welcomes changing requirements, even late in development

Agile Process Flow

1. Requirement gathering (User stories)
2. Planning (Sprint planning)
3. Design & Development
4. Testing
5. Review & Feedback
6. Deployment
7. Continuous improvement (Retrospective)

Common Agile Process Models

- Scrum
- Extreme Programming (XP)
- Kanban
- Lean
- Crystal

- In 1980's the **heavy weight, plan based software** development approach was used to develop any software product.
- In this approach too many things are done which were not directly related to software product being produced.
- This approach was rigid. That means if requirements get changed, then rework was essential. Hence new methods were proposed in 1990's which are known as agile processes.
- The agile processes are the light-weight methods are **people-based** rather than plan-based methods.
- The agile process forces the development team to **focus** on **software** itself rather than design and documentation.
- The agile process believes in **iterative method.**
- The aim of agile process is to **deliver** the working model of software **quickly** to the customer. **For example :** Extreme programming is the best known of agile process.

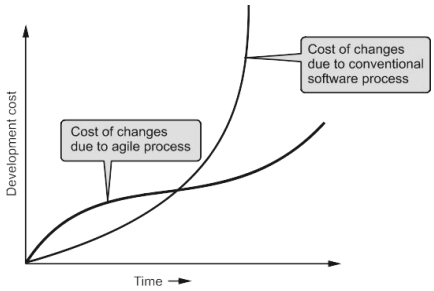


Fig. 1.9.1 Influence of software development approach on agile process

1.9.1 Agile Principles

There are famous 12 principles used as agility principles -

1. Satisfy the customer by early and continuous delivery of valuable software.
2. The changes in the requirements must be accommodated. Even though the changes occur late in the software development process, the agile process should help to accommodate them.
3. Deliver working software quite often. Within the shorter time span deliver the working unit.
4. Business people and developers must work together throughput the project.
5. Motivate the people who are building the projects. Provide the environment and support to the development team and trust them for the job to be done.
6. The working software is the primary measure of the progress of the software development.
7. The agile software development approach promote the constant project development. The constant speed for the development of the product must be maintained.
8. To enhance the agility there should be continuous technical excellence
9. The proper attention to be given to technical excellence and good design.
10. The simplicity must be maintained while developing the project using this approach.
11. The teams must be the self-organizing team for getting best architecture, requirements and design.
12. At regular intervals the team thinks over the issue of becoming effective. After the careful review the team members adjusts their behavior accordingly.

Advantages

- Faster delivery
- Higher customer satisfaction
- Reduced risk
- Better quality through continuous testing
- Improved team collaboration

Simple Example

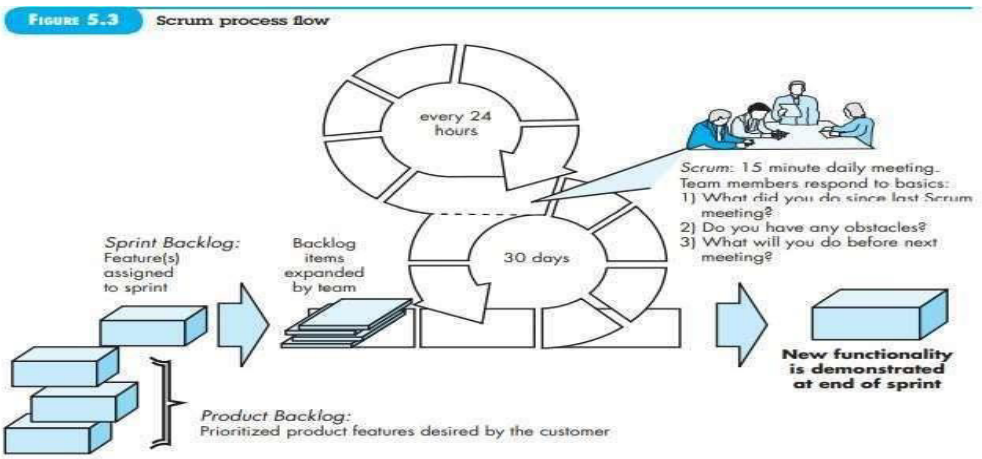
A mobile app is developed in 2-week sprints. After each sprint, a working version is shown to the client, feedback is collected, and changes are implemented in the next sprint.

Compare Conventional Methodology and Agile Methodology

Aspect	Conventional Methodology	Agile Methodology
Development style	Sequential and linear	Iterative and incremental
Requirement handling	Fixed at the beginning	Changes accepted anytime
Customer involvement	Limited	Continuous
Delivery	At the end of project	Frequent, small releases
Flexibility	Low	High
Documentation	Heavy documentation	Minimal but sufficient
Testing	After development phase	Continuous testing
Risk handling	High risk	Reduced risk
Team structure	Hierarchical	Self-organizing teams
Feedback	Late	Early and continuous
Examples	Waterfall, V-Model	Scrum, XP, Kanban

SCRUM

- Scrum is an Agile framework used to manage and develop complex products, especially in software development, through iterative and incremental delivery.
- Scrum is a lightweight agile framework that helps teams work collaboratively, deliver value in short cycles, and adapt quickly to change.
- Scrum principles are consistent with the agile manifesto and are used to guide development activities within a process that incorporates the following framework activities: requirements, analysis, design, evolution, and delivery.
- Within each framework activity, work tasks occur within a process called a sprint.
- The No. of Sprints within a framework activity will vary depending on product complexity and size.



Core Elements of Scrum

1. Scrum Roles

- **Product Owner**
  Defines requirements, manages the Product Backlog, and represents customer needs.
- **Scrum Master**
  Facilitates the Scrum process, removes impediments, and ensures Agile principles are followed.
- **Development Team**
  Cross-functional professionals who design, develop, and test the product.

2. Scrum Artifacts

- **Product Backlog** – List of all features, user stories, and requirements.
- **Sprint Backlog** – Tasks selected for the current sprint.
- **Increment** – Working product delivered at the end of each sprint.

3. Scrum Event

- **Sprint** – Fixed time-box (1–4 weeks).
- **Sprint Planning** – Decide what to build in the sprint.
- **Daily Scrum** – 15-minute daily stand-up meeting.
- **Sprint Review** – Demonstration of completed work.
- **Sprint Retrospective** – Team reflects and improves process.

Scrum Process Flow

1. Product Backlog creation
2. Sprint Planning
3. Sprint execution
4. Daily Scrum meetings
5. Sprint Review
6. Sprint Retrospective
7. Next Sprint

Advantages of Scrum

- Faster delivery
- Improved quality
- High transparency
- Better team collaboration
- Easy handling of changing requirements

Disadvantages

- Requires experienced team
- Not suitable for very large or fixed-scope projects
- Scope creep possible if not controlled

Simple Example

A web application is developed in 2-week sprints. After each sprint, a usable feature is delivered and reviewed with stakeholders.

OTHER AGILE FRAMEWORKS

The following are some of the methods to implementations of agile:

- Extreme Programming(XP)
- Dynamic Systems Development Method(DSDD)
- Agile Modeling (AM)
- Agile Unified Process (AUP) Lean
- Kanban

1.10 Extreme Programming

AU : May-19, Marks 8

Extreme Programming (XP) is one of the best known agile process. It is suggested by Kent Beck in 2000.

1.10.1 XP Values

Beck defined the set of five values that serve as a basis for the work performed in XP. These values are -

1. **Communication :** The effective communication must be established between software developers and stakeholders in order to convey the important concepts and to get the important feedback.
2. **Simplicity :** XP focuses on the current needs instead of futures needs to incorporate in the design. Hence the XP believes that the Software design should be simple.
3. **Feedback :** The feedback for the software product can be obtained from the developers of the software, customers, and other software team members.
4. **Courage :** The strict adherence to certain XP practices require courage. The agile XP team must be disciplined to design the system today, recognize the future requirements and make the changes dramatically as per the demand.
5. **Respect :** By following the above states XP values the agile team can win the respect of stakeholders.

1.10.2 Process

- The extreme programming process is explained as follows -
- Customer specifies and priorities the system requirements. Customer becomes one of the important members of development team. The developer and customer together prepare a **story-card** in which customer needs are mentioned.
- The developer team then aims to implement the scenarios in the story-card.
- After developing the story-card the development team breaks down the total work in **small tasks**. The efforts and the estimated resources required for these tasks are estimated.
- The customer priorities the stories for implementation. If the requirement changes then sometimes unimplemented stories have to be discarded. Then release the complete software in **small** and frequent **releases**.
- For accommodating new changes, **new story-card** must be developed.
- Evaluate, the system along with the customer.
- This process is demonstrated by the following Fig. 1.10.1.

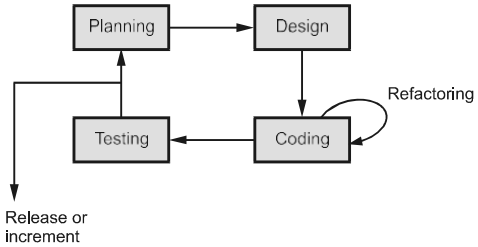


Fig. 1.10.1 XP process

- Various rules and practices used in extreme programming are as given below –

	XP Principle	Description
Planning	User story-cards	Instead of creating a large requirement document user stories are written by the customer in which what they need is mentioned.
	Release planning	A release plan for overall project is prepared from which the iteration plan can be prepared for individual iteration.
	Small releases	The developer breaks down the user stories into small releases and a plan for releasing the small functionalities is prepared.
	Iterative process	Divide the development work into small iterations. Keep the iteration of nearly constant length. Iterative development helps in quick or agile development.
	Stand up meetings	The stand up meetings must be conducted for the current outcomes of the project.
Designing	Simple design	Simple design always takes less time than the complex design. It is always good to keep the things simple to meet the current requirements.
	Spike solution	For answering the tough technical problems create the spike solutions. The goal of these solutions should be to reduce the technical risks.
	Refactoring	Refactoring means reductions in the redundancy, elimination of unused functionalities, redesign the obsolete designs. This will improve the quality of the project.

Coding	Customer availability	The most essential requirement of the XP is availability of the customer. In Extreme programming the customer not only helps the developer team but it should be the part of the project.
	Paired programming	All the code to be included in the project must be coded by groups of two people working at the same computer. This will increase the quality of coding.
	Collective code ownership	By having collective code ownership approach the everyone contributes new ideas and not any single person becomes the bottleneck of the project. Anyone can change any line of code to fix a bug or to refactor.
Testing	Unit testing	The test-first development is done in XP. The test framework that contains the automated test case suite is used to test the code. All the code must be tested using unit testing before its release.
	Continuous integration	As soon as one task is finished integrate it into the whole system. Again after such integration unit testing must be conducted.
	No overtime	Working overtime lose the spirit and motivation of the team. Conduct the release planning meeting to change the project scope or to reschedule the project.

As mentioned earlier in Extreme Programming instead of creating large requirement documents the user story-card is prepared. For example : If a project is for creating a banking software then the sample story card can be prepared as follows -

1. Customer makes the balance enquiry.
2. Customer withdraws some amount.
3. Customer deposits some amount.

Banking software checking balance, depositing and withdrawing
When the customer wants to **check the balance** then he just enters his account number and the total amount present in his account is displayed to him. Then customer can **withdraw** some amount from his account. The limit of amount to be withdrawn must be specified. If the customer crosses that limit then he must be warned. The amount withdrawn must be deducted from the available balance. The customer can **deposit** the desired amount in his account. The deposited amount must be added to the available balance.

Note that, the user story is broken into various tasks and simple classes must be created for these tasks. From above example it is clear that the useful class in the project could be *'account'* . Various data attributes can be *account_no, customer name, address, balance* and the useful functions can be *deposit(), withdraw(), check_balance().*

Each task can be separately tested using the automated test cases.

1.10.3 Industrial XP

- The industrial XP (IXP) can be defined as the organic evolution of XP. It is customer centric. It has expanded role for customers and advanced technical practices.
- Various new practices that are appended to XP to create IXP are as follows -

1. **Readiness Assessment :** In this assessment, following issues are assessed -
   i. Proper environment must be available to support XP.
   ii. Team should contain appropriate and skilled stakeholder
   iii. The organization should support quality programs and continuous improvement.
   iv. The organizational culture should support new values of agile team.
2. **Project Community:** Skilled and efficient people must be chosen as the agile team members for the success of the project. The team is referred as the **community** when extreme programming approach is considered. The project **community** consists of **technologies**, **customers**, and **other stakeholders** who play the vital role for the success of the project. The **role** of the community members must be **explicitly defined**.
3. **Project Chartering:** Project chartering means **assessing** the justification for the project as a business application. That means, the IXP team assess whether the project satisfies the goals and objectives of the organization.
4. **Test Driven management :** For assessing the state of the project and its progress the industrial XP needs some **measurable criteria**. In test driven management the project is tested with the help of these measurable criteria.
5. **Retrospectives :** After delivering the software increment, the **specialized review** is conducted which is called as **retrospective**. The intention of retrospectives is to improve the industrial XP process.
6. **Continuous learning :** The team members are inspired and encouraged to learn new methods and techniques that can **improve the quality** of the product.

2. Kanban

A visual workflow management framework.
Key Features:

- Kanban board (To Do – In Progress – Done)
- Work-in-progress (WIP) limits
- Continuous delivery (no fixed sprints)

Best suited for: Maintenance and support projects.

3. Lean Software Development

Derived from Lean manufacturing principles.
Principles:

- Eliminate waste
- Build quality in
- Deliver fast
- Continuous improvement

Best suited for: Process optimization and efficiency-driven teams.

4. Crystal

A family of Agile methods focused on people and communication.
Key Features:

- Minimal documentation
- Face-to-face communication
- Tailored based on team size and project criticality

Best suited for: Small to medium teams.

5. Feature-Driven Development (FDD)
Focuses on designing and building features.
Key Steps:

- Develop overall model
- Build feature list
- Design by feature
- Build by feature

Best suited for: Large-scale, feature-centric projects.

6. Dynamic System Development Method (DSDM)
An Agile project delivery framework emphasizing time and cost control.

Key Principle:

- Fix time and cost, vary scope

Best suited for: Business-critical applications.

Summary Table

Framework	Key Focus
XP	Technical excellence
Kanban	Workflow visualization
Lean	Waste reduction
Crystal	People-centric
FDD	Feature-based development
DSDM	Time-boxed delivery

Kanban

- The Japanese word "kanban", meaning **"visual board" or a "sign"**, has been used in the sense of a process definition since the 1950s. It was first developed and applied by **Toyota** as a **scheduling system for just-in-time manufacturing**. On the other hand, the capitalized term "Kanban" is known and associated with the emergence of the "Kanban Method," which was first defined in 2007.
- Kanban is an Agile framework used to **visualize work, limit work in progress (WIP), and improve flow for continuous delivery**.
- Kanban is a visual Agile method that manages work by displaying tasks on a Kanban board, enabling teams to deliver work continuously without fixed iterations.
- Kanban is an Agile framework that uses a **visual board to manage and optimize the flow of work through continuous delivery**.
- Kanban is a workflow management method for **defining, managing and improving services** that deliver knowledge work. It aims to help you visualize your work, maximize efficiency, and improve continuously.

The Kanban Board

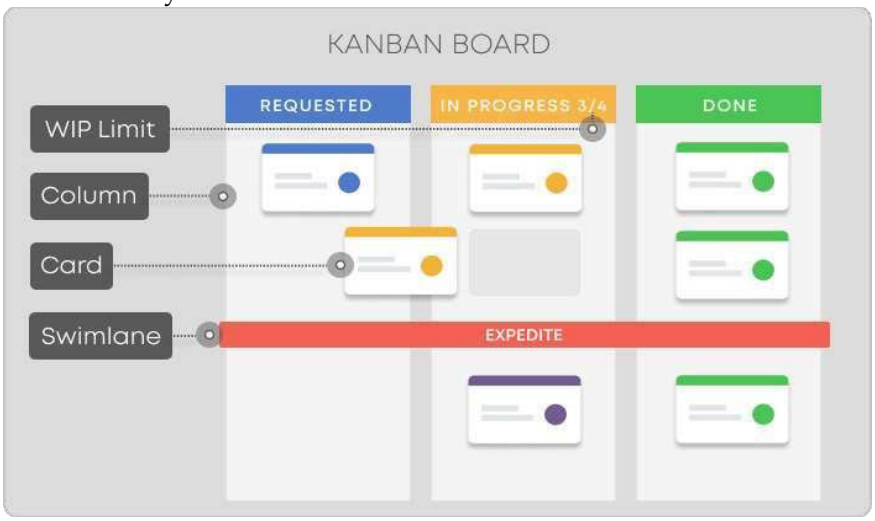
A Kanban board is a tool for workflow visualization and one of the Kanban method's key components.

Visualizing your workflow and tasks on a Kanban board helps you better understand your processes and gain an overview of your workload. With this new level of

transparency, you will quickly identify problematic work stages, and by improving those, your team will soon work more efficiently.

Main Components of the Kanban board

Kanban boards use Card, Column, Swimlanes, and WIP Limits to enable teams to visualize and manage their workflows effectively. Let us introduce you to the main components more closely:



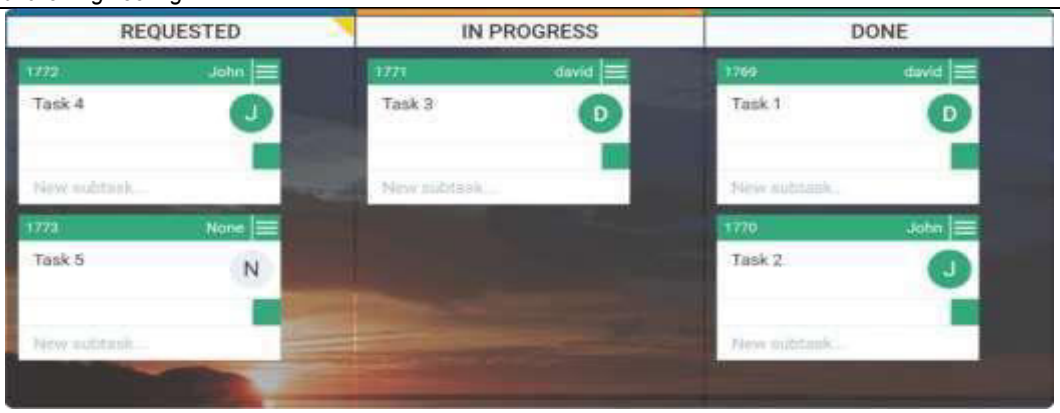
Kanban Cards – This is the visual representation of tasks. Each card contains information about the task and its status, such as deadline, assignee, description, etc.

Kanban Columns – Each column on the board represents a different stage of your workflow. The cards go through the workflow until their full completion.

Work-in-Progress Limits – They restrict the maximum amount of tasks in the different stages of the workflow. Limiting WIP allows you to finish work items faster by helping your team focus only on current tasks.

Kanban Swimlanes – These are horizontal lanes you can use to separate different activities, teams, classes of service, and more.

If you are new to the method, you may start with a basic Kanban board structure and split it into three primary sections that show different workflow stages.



Kanban metrics

Kanban uses specific metrics to measure team capacity and estimate project length.

Team velocity defines how many tasks a team can deliver in a given period of time, for example a week or iteration.

Lead and Cycle time defines the average time it takes to complete a task. **Lead** time is used to understand how long a client has to wait for their product and **cycle time** is used to understand how fast the team produces a product.

Actionable Agile metrics use cycle time to better predict when each project item is going to be finished.

Key Principles of Kanban

1. Visualize the workflow
2. Limit Work in Progress (WIP)
3. Manage flow
4. Make process policies explicit
5. Implement feedback loops
6. Improve collaboratively and continuously

Kanban Board

A typical Kanban board has columns such as:

- To Do
- In Progress
- Testing
- Done

Each task is represented as a Kanban card.

Key Features

- No fixed sprints
- Continuous delivery

- Pull-based system
- Flexible priorities
- Reduced bottlenecks

Advantages

- Easy to understand and implement
- High visibility of work
- Improves efficiency
- Ideal for support and maintenance work
- Reduces multitasking

Limitation

- No fixed deadlines
- Requires discipline to follow WIP limits
- Less suitable for large, long-term planning

Kanban vs Scrum (Quick View)

Aspect	Kanban	Scrum
Iterations	Continuous	Fixed sprints
Roles	No mandatory roles	Defined roles
Changes	Anytime	Limited during sprint
Board	Workflow-based	Sprint-based

DevOps

- DevOps is a culture, set of practices, and tools that integrates software development (Dev) and IT operations (Ops) to enable faster, reliable, and continuous delivery of software.
- DevOps is an approach that emphasizes collaboration, automation, continuous integration, and continuous delivery (CI/CD) to shorten the software development life cycle and improve deployment quality.

Core Objectives

- Faster software delivery
- Continuous deployment
- Improved collaboration
- Higher reliability and stability
- Automation of processes

DevOps Lifecycle

1. Plan – Requirement analysis and planning
2. Code – Application development
3. Build – Compile and package code
4. Test – Automated testing
5. Release – Prepare for deployment
6. Deploy – Deploy to production
7. Operate – Maintain system stability
8. Monitor – Continuous monitoring and feedback

Key DevOps Practices

- Continuous Integration (CI)
- Continuous Delivery/Deployment (CD)
- Infrastructure as Code (IaC)
- Automated testing
- Continuous monitoring
- Configuration management

Common DevOps Tools

- Version Control: Git
- CI/CD: Jenkins, GitHub Actions
- Containers: Docker
- Orchestration: Kubernetes
- Configuration: Ansible, Chef
- Monitoring: Prometheus, Grafana

Benefits

- Faster release cycles
- Reduced deployment failures
- Improved collaboration between teams
- Quick recovery from failures
- High customer satisfaction

DevOps vs Agile

Aspect	Agile	DevOps
Focus	Development	Dev + Operations
Goal	Fast development	Fast & reliable deployment
Scope	Software creation	End-to-end delivery

UNIT II
REQUIREMENTS AND UNIFIED MODELING

Requirements Engineering – Establishing the Groundwork: Nonfunctional Requirements – Requirements Gathering – Developing Use Cases – Negotiating and Validating Requirements. Unified Modeling Language – Introduction – Static and Dynamic Models – Modelling – Introduction to the UML – UML Diagrams – UML Class Diagrams – Use-Case Diagram – UML Dynamic Modelling

2.1 Software Requirements

What is requirement engineering ?

Requirement engineering is the process of

- establishing the services that the customer requires from a system
- and the constraints under which it operates and is developed.

The requirements themselves are the descriptions of the system services and constraints that are generated during the requirements engineering process.

What is a requirement ?

A requirement can range from a high-level abstract statement of a service or of a system constraint to a detailed mathematical functional specification.

The requirement must be open to interpretation and it must be defined in detail.

Types of requirements

The requirements can be classified as

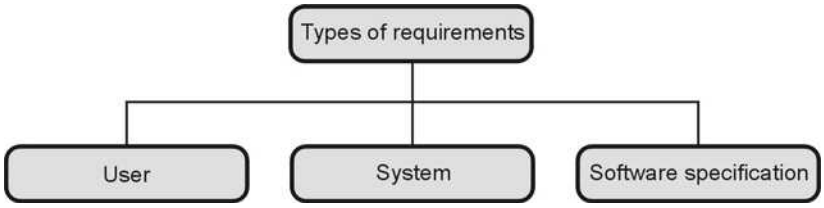


Fig. 2.1.1 Types of requirements

- **User requirements**

It is a collection of statements in natural language plus description of the services the system provides and its operational constraints. It is written for customers.

- **System requirements**

It is a structured document that gives the detailed description of the system services. It is written as a contract between client and contractor.

- **Software specification**

It is a detailed software description that can serve as a basis for design or implementation. Typically it is written for software developers.

2.2 Functional and Non Functional Requirements

AU : Dec.-13,17, May-13, Marks 11

Software system requirements can be classified as functional and non functional requirements.

2.2.1 Functional Requirements

- Functional requirements should describe all the required functionality or system services.
- The customer should provide statement of service. It should be clear how the system should react to particular inputs and how a particular system should behave in particular situation.
- Functional requirements are heavily dependant upon the type of software, expected users and the type of system where the software is used.
- Functional user requirements may be high-level statements of what the system should do but functional system requirements should describe the system services in detail.
- For example : Consider a library system in which there is a single interface provided to multiple databases. These databases are collection of articles from different libraries. A user can search for, download and print these articles for a personal study.

From this example we can obtain functional Requirements as–

1. The user shall be able to search either all of the initial set of databases or select a subset from it.
2. The system shall provide appropriate viewers for the user to read documents in the document store.
3. A unique identifier (ORDER_ID) should be allocated to every order. This identifier can be copied by the user to the account's permanent storage area.

2.2.1.1 Problems Associated with Requirements

- **Requirements imprecision**

1. Problems arise when requirements are not precisely stated.
2. Ambiguous requirements may be interpreted in different ways by developers and users.
3. Consider meaning of term 'appropriate viewers'.

- **User intention -** special purpose viewer for each different document type;
- **Developer interpretation -** Provide a text viewer that shows the contents of the document.

- **Requirements completeness and consistency -**

The requirements should be both complete and consistent. *Complete* means they should include descriptions of all facilities required. *Consistent* means there should be no conflicts or contradictions in the descriptions of the system facilities.

Actually in practice, it is impossible to produce a complete and consistent requirements document.

2.2.2 Non Functional Requirements

- The non functional requirements define system properties and constraints.
- Various properties of a system can be : Reliability, response time, storage requirements. And constraints of the system can be : Input and output device capability, system representations etc.
- Process requirements may also specify programming language or development method.
- Non functional requirements are more critical than functional requirements. If the non functional requirements do not meet then the complete system is of no use.

2.2.2.1 Types of Non Functional Requirements

The classification of non functional requirements is as given below. (See Fig. 2.2.1 on next page)

Product requirements

These requirements specify how a delivered product should behave in a particular way. For instance: execution speed, reliability.

Organizational requirements

The requirements which are consequences of organizational policies and procedures come under this category. For instance: process standards used implementation requirements.

External requirements

These requirements arise due to the factors that are external to the system and its development process. For instance : interoperability requirements, legislative requirements.

In short, non functional requirements arise through

i) User needs
ii) Because of budget constraints

iii) Organizational policies

iv) The need for interoperability with other software or hardware systems

v) Because of external factors such as safety regulations.

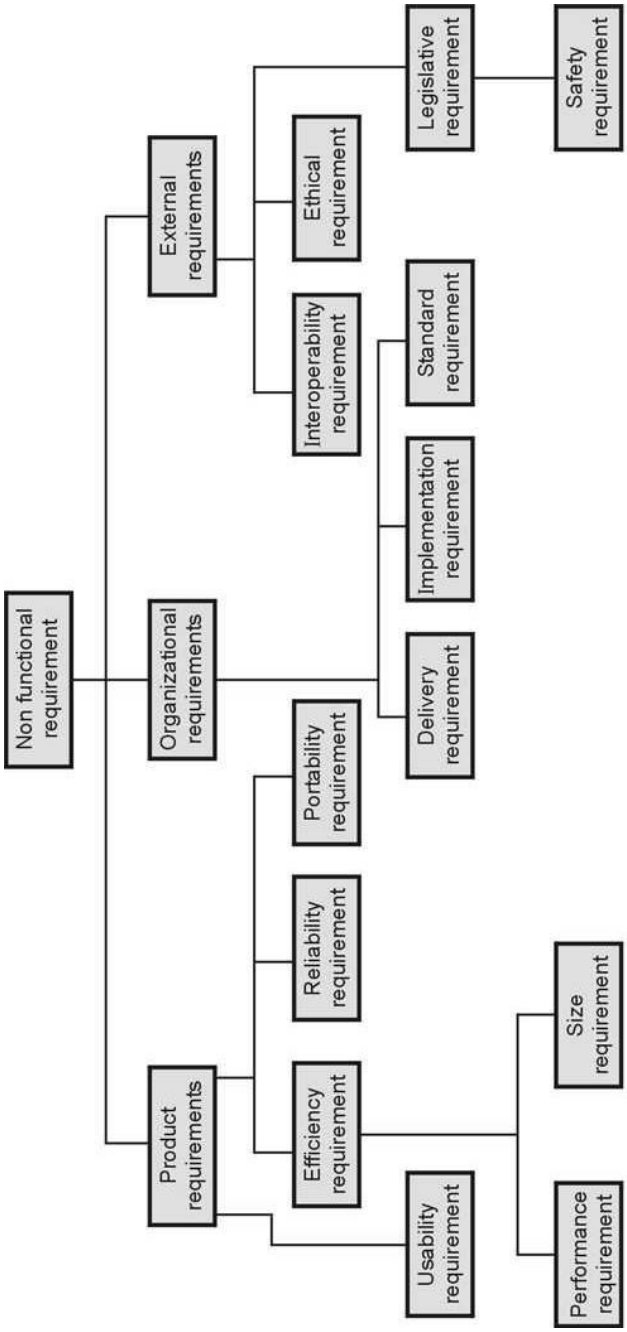


Fig. 2.2.1 Types of non functional requirement

Metrics used for specifying the non functional requirements

Property	Metric
Speed	Events per response time processed transactions per second.
Size	Kilo bytes.
Reliability	Mean time to failure. Rate of failure. Occurrence availability.
Robustness	Time to restart after failure. Probability of events causing failure.
Portability	Number of target statements.

2.2.2.2 Domain Requirements

- Domain requirements are derived from the application domain of the system instead of specific user needs.
- These requirements make use of domain terminologies specific to the existing domain concept.
- The domain requirements may be in the form of new functional requirements, constraints on existing functional requirement or guidance on how to carry out certain computation.
- These are the specialised requirements and hence software engineers find it difficult to co-relate the domain requirements with the system requirements.
- It is important to specify the domain requirements otherwise the system will not work properly.
- **Example : Domain requirements for the library system.**
- There should be user interface for handling the databases. These interfaces should be according to some international standard.
- If there is copyright restriction on some document then it should get printed locally on the server. The copies of such document should not get created.

2.2.3 Difference between Functional and Non Functional Requirements

Functional requirements	Non functional requirements
The functional requirements specify the features of the software system	The non functional requirements specify the properties of the software system.

Functional requirements describe what the product must do	Non functional requirements describe how the product should perform.
The functional requirements specify the actions with which the work is concerned.	The non functional requirements specify the experience of the user while using the system.
Example : For a library management system, allowing user to read the article online is a functional requirement.	Example : For a library management system, for a user who whishes to read the article online must be authenticated first.

Metrics used for specifying Non Functional Requirements

The metrics used for specifying the non functional requirements are -

1. **Reliability :** The application should be highly reliable and it should generate all the updated information in correct order.
2. **Availability :** Any information must be quickly available from computer to the authorized user.
3. **Security :** The application must be password protected. This feature is essential to avoid any unauthorized access to the application.
4. **Maintainability :** The application should be maintained in such a manner that if any change in requirement occurs then it should be easily incorporated in an individual module.
5. **Extensibility :** The application should be maintained in such a manner that if any new requirement occurs then it should be easily incorporated in an individual module.
6. **Portability :** The application should be portable on the desired operating system.
7. **Reusability :** The application should have the ability that a segment of source code can be used again to add new functionalities with slight or no modification. Reusable modules and classes reduce implementation time.
8. **Resource utilization :** The application should make use of resources to its maximum capability.

Example 2.2.1 *For the requirement given below, identify stakeholders, functional and non-functional requirements*

A software is to be built that will control an Automated Teller Machine (ATM). The ATM machine services customers 24 × 7. ATM has a magnetic stripe reader for reading an ATM card, a keyboard and display for interaction with the customer, a slot for depositing envelopes, a dispenser for cash, a printer for printing receipts and a switch that allows an operator to start/stop a machine.

The ATM services one customer at a time. When a customer inserts an ATM card and enters the Personal Identification Number (PIN), the details are validated for each transaction. A customer can perform one or more transactions. Transactions made against each account are recorded so as to ensure validity of transactions.

If PIN is invalid, customer is required to re-enter PIN before making a transaction, If customer is unable to successfully enter PIN after three tries, card is retained by machine and customer has to contact bank.

The ATM provides the following services to the customer :

1) Withdraw cash in multiples of 100

2) Deposit cash in multiples of 100

3) Transfer amount between any two accounts.

4) Make balance enquiry.

5) Print receipt.

Each of the above transactions must be made within 60 seconds in case the time exceeds 60 seconds, then the transaction is cancelled automatically. Also, if the machine is not used for more than two minutes after entry of card, the card is retained by the machine. An operator panel with a key-operated switch allows an operator to start and stop the servicing of customers. When the switch is moved to the "off position, the machine will shut down, so that the operator may remove deposit envelopes and reload the machine with cash, blank receipts, etc. The operator is required to verify and enter the total cash on hand before starting the system from this panel. **AU : Dec.-13, Marks 11**

Solution : i) Stakeholders

1. Bank customer
2. Service operator
3. Hardware and software maintenance engineers
4. Database administrators
5. Banking regulators
6. Security administrator

ii) Functional requirements

1. There should be the facility for the Customer to insert a card.
2. The system should first validate card and PIN.
3. The system should allow the customer to deposit amount in the bank.
4. The system should dispense the cash on withdrawal.
5. The system should provide the printout for the transaction.
6. The system should make the record of the transactions made by particular customer.
7. On invalid PIN entry for three times the card should be retained by the system.
8. The cash withdrawal is allowed in multiple of 100.
9. The cash deposition is allowed in multiple of 100.
10. The customer is allowed to transfer amount between the two accounts.
11. The customer is allowed to know the balance enquiry.
12. The customer is allowed to get the printout for desired transaction.
13. The system should be efficient.

iii) Non functional requirements

1. Each of the transaction should be made within 60 seconds. If the time limit is exceeded ,then cancel the transaction automatically.
2. If there is no response from the bank computer after request is made within the minutes then the card is rejected with error message.
3. The bank dispenses money only after the processing of withdrawal from the bank. That means if sufficient fund is available in user's account then only the withdrawal request is processed.
4. Each bank should process the transactions from several ATM centers at the same time.
5. The machine should be loaded with sufficient fund in it.

Example 2.2.2 *List the stakeholders and all types of requirements for an online train reservation system.* **AU : Dec.-17, Marks 7**

Solution :

1) Passenger
2) Database Administrator
3) Booking Clerk
4) Bank

Functional Requirements

1) The online booking made by the customer should be associated with the account.
2) Booking confirmation must be sent to the user on specified contact details.
3) The system should provide a search option to the user so that user can search for required train and for required number of reservations.
4) The system should allow the user to make online payments.
5) The system should allow user to book for tickets.
6) The system should allow the user to cancel the booking and half of the amount paid by the customer must be refunded to him.

Non Functional Requirements

1) The system must be available to the user for 24*7.
2) The system should accept the payments via different payment methods like PayPal, Wallets, Cards, vouchers etc.
3) Use of captcha and encryption to avoid bots from booking tickets.
4) The response time of the system while searching, booking or cancellation operation should be very less.

Review Question

1. *Explain the metrics used for specifying non functional requirements.* **AU : May-13, Marks 8**

2.3 User Requirements

- The user requirements should describe functional and non functional requirements in such a way that they are understandable by system users who don't have detailed technical knowledge.
- User requirements are defined using natural language, tables and diagrams because these are the representations that can be understood by all users.

Various **problems** that can arise **in the requirement specifications** when requirements are given in natural language -

Lack of clarity

Sometimes requirements are given in ambiguous manner. It is expected that text should help in clear and precise understanding of the requirements.

Requirements confusion

There may be confusion in functional requirements and non functional requirements, system goals and design information.

Requirements mixture

There may be a chance of specifying several requirements together as a single requirement.

2.3.1 Guidelines for Writing User Requirements

- Prepare a standard format and use it for all requirements.
- Apply consistency in the language. Use
  - 'shall' for mandatory requirements
  - and 'should' for desirable requirements.
- The text which is mentioning the key requirements should be highlighted.
- Avoid the use of computer jargon (computer terminologies). It should be written in simple language.

For example

Consider a spell checking and correcting system of a word processor. The user requirements can be given in natural language as

1. The system should posses a traditional **word dictionary and user supplied dictionary.** It shall provide a user-activated facility which checks the spelling of words in the document against spellings in the system dictionary and user-supplied dictionaries.
2. When a word is found in the document which is not given in the dictionary, then the system should suggest 10 alternative words. These alternative words should be based on a match between the word found and corresponding words in the dictionaries.
3. When a word is found in the document which is not in any dictionary, the system should propose following options to user :

1. Ignore the corresponding instance of the word and go to next sentence.
2. Ignore all instances of the word.
3. Replace the word with a suggested word from the dictionary.
4. Edit the word with user-supplied text.
5. Ignore this instance and add the word to a specified dictionary.

2.4 System Requirements

AU : May-16, Marks 4

- System requirements are more detailed specifications of system functions, services and constraints than user requirements.

- They are intended to be a basis for designing the system.
- They may be incorporated into the system contract.
- The system requirements can be expressed using system models.
- The requirements specify what the system does and design specifies how it does.
- System requirement should simply describe the external behavior of the system and its operational constraints. They should not be concerned with how the system should be designed or implemented.
- For a complex software system design it is necessary to give all the requirements in detail.
- Usually, natural language is used to write system requirements specification and user requirements.

2.4.1 Structured Language Specification

- One of the method of writing requirements is by using structured natural language.
- All the requirements should be written in a **standard way** while using structured language specification.
- The **advantage** of specifying requirements using this method is that requirement become **understandable** and **expressive.**
- The only necessary thing while writing requirements using natural language is that some degree of **uniformity must be maintained.**
- Extra information can be added when the requirements are written using natural language. This information can be represented using tables or graphical models.
- One way of using graphical model is use of **sequence diagram.**
- The sequence diagram represents the sequence of actions that user performs while interacting the system.
- Example : Following is a sequence diagram for withdrawal of cash from ATM. (See Fig. 2.4.1 on next page)

Why requirement and design are inseparable ?

- A system architecture may be designed to structure the requirements.
- The system may inter-operate with other systems and that may generate design requirements.
- The use of a specific design may be a domain requirement.

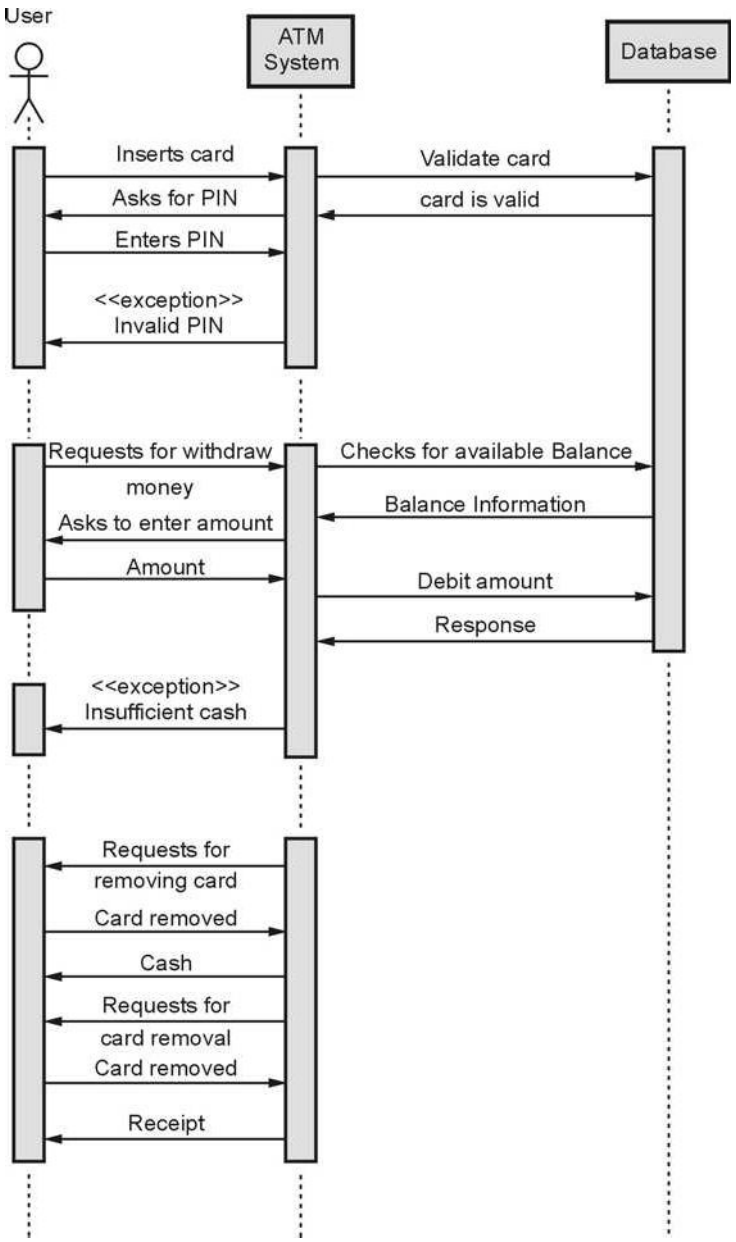


Fig. 2.4.1 Sequence diagram of ATM withdrawal

Example 2.4.1 *Differentiate between user and system requirements.* **AU : May-16, Marks 4**

Solution :

User Requirement	System Requirement
The focus of this types of requirements is on the problem domain.	The focus of this type of requirements is on solution domain.
These requirements describe what effects need to be achieved.	These requirements describe what software the software must do.
User requirement tell what application should do to satisfy users needs.	System requirements tell a system should have to be able to run program.
The user defines his/her requirements for the system. Hence it is not necessary to incorporate all the user requirements in the system contract.	These requirements must be incorporated into system contract.

2.5 Software Requirements Document

AU : Dec.-11, 19, May-13,14,16,18,19, Marks 16

The software requirements document is the specification of the system. It should include both a definition and a specification of requirements. It is not a design document. As far as possible, it should set of what the system should do rather than how it should do it.

Software Requirements Specification

The software requirements provide a basis for creating the Software Requirements Specifications (SRS).

The SRS is useful in estimating cost, planning team activities, performing tasks, and tracking the team's progress throughout the development activity.

Typically software designers use IEEE STD 830-1998 as the basis for the entire Software Specifications. The standard template for writing SRS is as given below.

Document Title

Author(s)
Affiliation
Address
Date
Document Version

1. Introduction

1.1 Purpose of this document

Describes the purpose of the document.

1.2 Scope of this document

Describes the scope of this requirements definition effort. This section also details any constraints that were placed upon the requirements elicitation process, such as schedules, costs.

1.3 Overview

Provides a brief overview of the product defined as a result of the requirements elicitation process.

2. General Description

- Describes the general functionality of the product such as similar system information, user characteristics, user objective, general constraints placed on design team.
- Describes the features of the user community, including their expected expertise with software systems and the application domain.

3. Functional Requirements

This section lists the functional requirements in ranked order. A functional requirement describes the possible effects of a software system, in other words, *what* the system must accomplish. Each functional requirement should be specified in following manner

- **Short, imperative sentence stating highest ranked functional requirement.**

1. **Description**
   A full description of the requirement.
2. **Criticality**
   Describes how essential this requirement is to the overall system.
3. **Technical issues**
   Describes any design or implementation issues involved in satisfying this requirement.
4. **Cost and schedule**
   Describes the relative or absolute costs of the system.
5. **Risks**
   Describes the circumstances under which this requirement might not able to be satisfied.

6. **Dependencies with other requirements**

   Describes interactions with other requirements.

7. **.....any other appropriate**

4. Interface Requirements

This section describes how the software interfaces with other software products or users for input or output. Examples of such interfaces include library routines, token streams, shared memory, data streams, and so forth.

- **4.1 User Interfaces**

  Describes how this product interfaces with the user.

  - **4.1.1 GUI**

    Describes the graphical user interface if present. This section should include a set of screen dumps to illustrate user interface features.

  - **4.1.2 CLI**

    Describes the command-line interface if present. For each command, a description of all arguments and example values and invocations should be provided.

  - **4.1.3 API**

    Describes the application programming interface, if present.

- **4.2 Hardware Interfaces**

  Describes interfaces to hardware devices.

- **4.3 Communications Interfaces**

  Describes network interfaces.

- **4.4 Software Interfaces**

  Describes any remaining software interfaces not included above.

5. Performance Requirements

Specifies speed and memory requirements.

6. Design Constraints

Specifies any constraints for the design team such as software or hardware limitations.

7. Other Non-Functional Attributes

Specifies any other particular non functional attributes required by the system. Such as :

7.1 Security

7.2 Binary Compatibility

7.3 Reliability

7.4 Maintainability

7.5 Portability

7.6 Extensibility

7.7 Reusability

7.8 Application Compatibility

7.9 Resource Utilization

7.10 Serviceability

... others as appropriate

8. Operational Scenarios

This section should describe a set of scenarios that illustrate, from the user's perspective, what will be experienced when utilizing the system under various situations.

9. Preliminary Schedule

This section provides an initial version of the project plan, including the major tasks to be accomplished, their interdependencies, and their tentative start/stop dates.

10. Preliminary Budget

This section provides an initial budget for the project.

11. Appendices

11.1 Definitions, Acronyms, Abbreviations

Provides definitions terms, and acronyms, can be provided.

11.2 References

Provides complete citations to all documents and meetings referenced.

2.5.1 Characteristics of SRS

Various characteristics of SRS are

- Correct - The SRS should be made up to date when appropriate requirements are identified.
- Unambiguous - When the requirements are correctly understood then only it is possible to write an unambiguous SRS.

- Complete - To make the SRS complete, it should be specified what a software designer wants to create a software.
- Consistent - It should be consistent with reference to the functionalities identified.
- Specific - The requirements should be mentioned specifically.
- Traceable - What is the need for mentioned requirement? This should be correctly identified.

Example 2.5.3 *Prepare a software requirement specification document for "Library Management System"*

AU : Dec.-19, Marks 15

Solution :

1. Introduction

1.1 Purpose

This document gives the detail functional and non functional requirements for Library management system. Thee purpose of this document is that the requirements mentioned in it should be utilized by the software developer to implement the system.

This SRS will be used by the software engineers who are constructing the system and the users who will use the system. The hotel end users will be able to use this SRS as a test to see if the software engineers will be constructing the system to their expectations.

1.2 Scope

The library management system will automate the major library operations. Various subsystems will be student accounts management system, Book issuing system, Stock maintenance systems and fine calculation and management system.

1.3 Definitions, Acronyms and Abbreviations

- LMS - Library Management System
- SRS - Software Requirement Specification
- GUI - Graphical User Interface
- Stakeholder - The person who will participate in this system. For example Student, Staff, Librarian.

1.3 Overview

This system provides an easy solution for the students, staff and Librarian for accessing and managing the books in the library.

2. Overall Description

The Library management system is developed for handling the activities for various users such as Student, staff, librarian and library staff.

2.1 Product Perspective

This is a standalone system.

2.2 Hardware Interfaces

This system is placed in the Personal Computer in the library.

2.3 Software Interfaces

There are various databases in the system. These databases can be created in Oracle or MySQL. The hotel books and Student/Staff information is maintained by the LMS.

The book database include the Title of the book, ISBN number, publisher, edition, number of copies, price,status(Old or new). The Student/staff database will include information such as first name, last name, address, phone number, designation(for staff) or class(for student), name of books issued, Library Card number, expects date of book return, fine to be paid(if any).

3. Functional Requirements

The functional requirement will define the fundamental functioning that the system should perform.

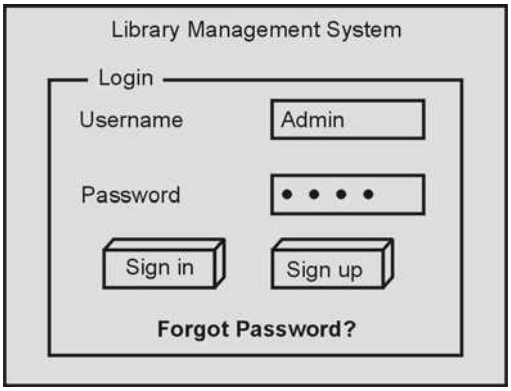
1. The system should record all the details of the student, staff and book.
2. The system should display the search results if the user searches for some book or article.
3. The system should allow to select the book for issuing.
4. The system should record the expected issue date and time of book.
5. The system should allow the modification in the student/staff/book information without reentering the entire information.
6. The system should allows to create an account for the new student.
7. The system should allow to cancel the account if the student leaves the college.
8. The system should display the availability of book or magazine.
9. The system shall allow to assign password to the users.
10. The system should keep track of the purchased items stock.
11. The member is allowed to reserve a book if it is currently unavailable.
12. The system should generate daily or monthly report for the stock.

13. The system should allow addition of book/magazine records if new books are purchased.
14. The system should allow deletion of book/magazine records if the books are old and in poor condition.

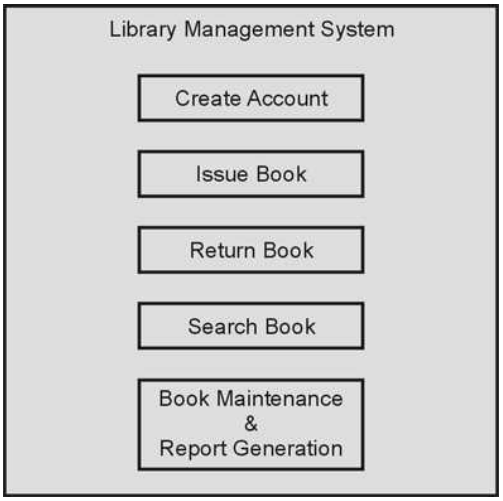
4. Interface Requirements

4.1 GUI

GUI 1 : The login screen will be displayed so that the stakeholder of the system will enter the user name and password.



GUI 2 : The main menu will be displayed for Create Account, Search, Issue/Return Book, Book maintenance and Report generation.



GUI 2 : When the Create Account button is clicked then the form will be displayed. This form will allow the user to enter the Student/Staff details such as last name, first name, phone number, class and so on.

GUI 3 : If the user selects for the Search then the user will be allowed to search for the desired book.

4.2 Hardware Interfaces

The system should be embedded in the Computers in the library.

4.3 Software Interface

The system must be interfaced with Oracle or Access database.

5. Performance Requirements

The performance requirement defines the response time for system functionality.

The load time for the GUI should not be more than four seconds.

The log in must be verified within 7 seconds.

The search query should be processed with in 3 seconds and response must be given.

6. Design Constraints

The system must be designed as a standalone system and must be run on window based system. The system can be developed in Java or Visual Basic. The database can be implemented in Oracle or MySQL.

7. Other Non Functional Attributes

7.1 Security

The System administrator/Librarian must provide password to log on the system. The password can be changed by the system administrator/Librarian. The Student should be allowed to the access the book database for searching of book. He/She is not allowed to modify the database.

The student/Staff is allowed to change their passwords for security purpose.

7.2 Reliability

The system must be reliable to prevent any unauthorized access.

7.3 Availability

The system should be available during the College hours.

7.4 Maintainability

There should be the facility to add/modify information about the Books, Student and Staff.

8. Operational Scenarios

If a new students arrives in the library for membership then the librarian creates an account; issues him the library Card, User name and password.

If the student already has an account, he searches for the desired book/magazine. If the book is available then he makes the request to the librarian for issuing the book. The librarian issues the book and updates the database.

If the book is not available then the member reserves the book. On availability of the book, it is issued to the customer.

If the student/staff does not return the book on the specified date then he has to pay the fine.

The librarian updates the stock and the system generates the monthly report. The old and outdated books are removed from the library and accordingly the stock must be updated.

If new purchase is made then also librarian is allowed to update the book database.

9. Preliminary Schedule

The system must be implemented within 6 months.

Review Questions

1. *Show the template of IEEE standard software requirements document.* **AU : May-13, Marks 8**
2. *What are the components of the standard structure for the software requirements document ? Explain in detail.* **AU May-14, Marks 8, May-18, Marks 16**
3. *Write the software requirement specification for a system of your choice.* **AU : May-14, Marks 8**
4. *Explain the organization of SRS and highlight the importance of each subsection.* **AU : May-16, Marks 8**

2.6 Requirement Engineering Process

AU : May-15, 17, Dec.-15,19, Marks 16

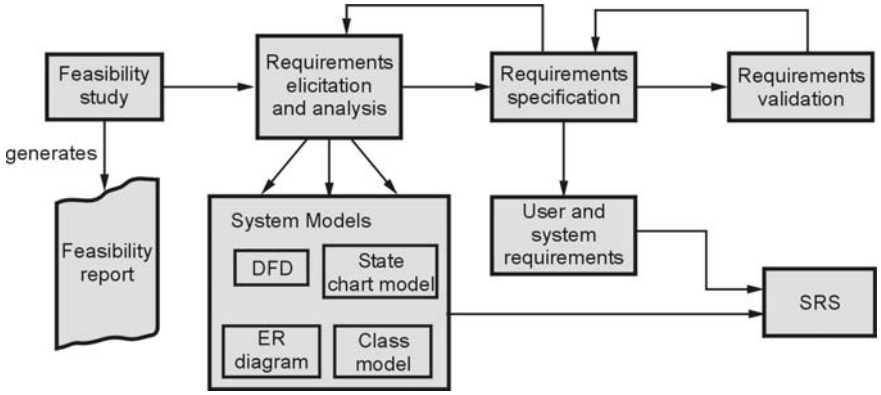


Fig. 2.6.1 Requirement engineering process

- A Requirement Engineering is a process in which various activities such as **discovery, analysis** and **validation** of system requirements are done.

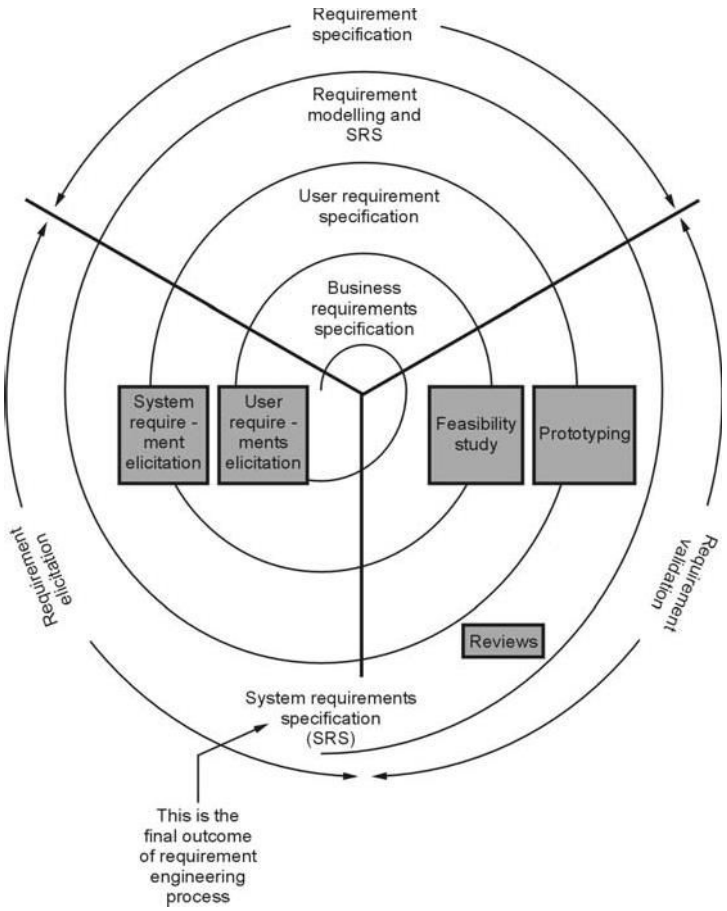


Fig. 2.6.2 Spiral model of requirements engineering process

- It begins with feasibility study of the system and ends up with requirement validation. Finally the requirement document has to be prepared.
- This process is a three stage activity where the activities are arranged in the iterative manner. In the early stage of this process most of the time is spent on understanding the system by understanding the high-level non functional requirements and user requirements. The spiral model of requirement engineering process is as shown in Fig. 2.6.2.

Requirement engineering process performs following **seven distinct functions** -

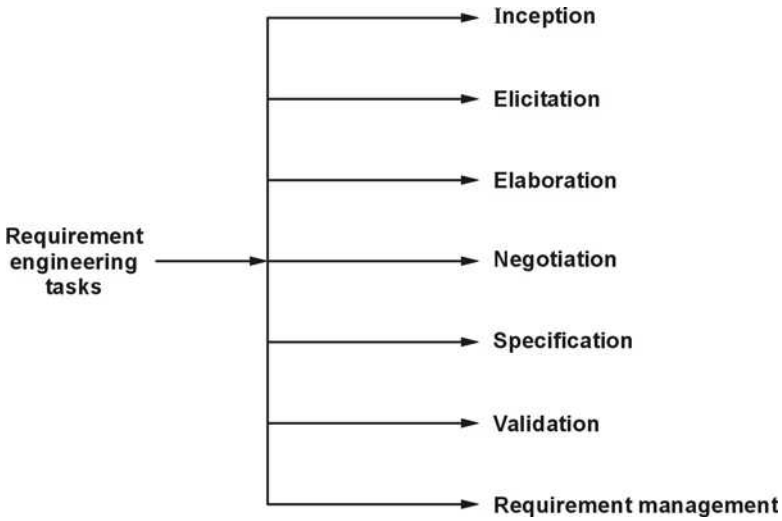


Fig. 2.6.3 Requirement engineering tasks

Let us now discuss these tasks in detail -

Inception : The inception means specifying the **beginning** of the software project. Most of the software projects get started due to business requirements. There may be potential demand from the market for a particular product.

The purpose of inception is to -

1. Establish the basic understanding of the project.
2. Find out all possible solutions and to identify the nature of the solution.
3. Establish an effective communication between developer and the customer.

Elicitation : Requirements elicitation means **requirements discovery.** Requirements elicitation is very difficult task.

Following are the reasons for : ***why it is difficult to understand customer wants? -***

1. Customer sometimes is unable to specify the **scope of the project**. Sometimes customers specify too many technical details and this may increase the confusion.

2. There is difficulty in **understanding the problem.** Sometimes customer could not decide what are their needs and wants. Sometimes they have got poor understanding of capabilities and limitations of the existing computing environment.

   Sometimes customers find it **difficult to communicate** with the system engineer about their needs. Sometimes customers may have got **some conflicting requirements.** This ultimately results in specifying **ambiguous requirements.**

3. As project progresses the needs or requirements of the customers changes. This creates a problem of volatility.

To overcome these problems the requirements gathering must be done very systematically.

Elaboration : Elaboration is an activity in which the information about the requirements is **expanded** and **refined**. The **goal** of elaboration activity is to prepare a technical model of software functions, features and constraints. The elaboration consists of several **modelling and refinement tasks**. In this process several **user scenarios** are created and refined. During elaboration, each user scenario is parsed and various **classes** are identified. These classes are nothing but the business entities that are visible to end user. Then the **attributes** and **services** (functions) of these classes are defined. Then the relationship among these classes is identified. Finally the **analysis model** gets developed during the elaboration phase.

Negotiation : Sometimes customer may **demand for more** than that is achieved or there are certain situations in which customer demands for something which cannot be achieved in **limited business resources**. To handle such situations requirement engineers must convince the customers or end users by solving various **conflicts**.

Specification : A specification can be a **written document, mathematical** or **graphical model, collection of use case scenarios** or may be the **prototypes.** There is a need to develop a **standard specification** in which requirements are presented in consistent and understandable manner.

Validation : Requirement validation is an activity in which requirement specification is analyzed in order to ensure that the requirements are specified unambiguously. If any inconsistencies, omissions and errors are identified then those are corrected or modified during the validation.

Requirement management : Requirements management is the process of managing changing requirements during the requirements engineering process and system development.

2.7 Feasibility Studies

AU : May-17,Dec.-17, Marks 8

Definition : *A feasibility study is a study made to decide whether or not the proposed system is worthwhile.*

The **focus** of feasibility study is to check

- If the system contributes to **organizational objectives.**
- If the system can be engineered using **current technology.**
- If the system is within the given **budget.**
- If the system can be **integrated** with other useful systems.

The implementation of feasibility study is based on the information assessment (what is required), information collection and report writing.

While performing the feasibility study, following questionnaires to the people in the organization should be asked -

1. What if the system wasn't implemented ?
2. What are current process problems ?
3. How will the proposed system help ?
4. What will be the integration problems ?
5. Is new technology needed? With what skills ?
6. What facilities must be supported by the proposed system ?

Feasibility study should be done with the help of project managers who is going to handle that particular project, software engineers who are about to develop that system, technical experts and customers who will be using the system. Typically the feasibility study should be completed within **two to three weeks**.

Finally the **feasibility study report** has to be prepared.

Types of Feasibility Study

1. **Technical Feasibility :** It is the measure of technical resources such as hardware components, software techniques and skilled persons.
2. **Economical Feasibility :** It is the measure whether finances or funds are available for proposed system.
3. **Operational Feasibility :** It is a measure of how well the solution of problems or a specific alternative solution will work in the organization.
4. **Schedule Feasibility :** It is the establishment of time limit for completion of the project. This kind of feasibility is dependent upon available manpower and economical support for the project.

Example *Develop complete use case for the following : Making a withdrawal from ATM in a bank*

Solution :

Use Case	ATM system
Primary Actor	Customer
Goal in Context	To monitor all the functionalities required to establish the session
Preconditions	ATM system has to be programmed and as to recognise and validate the PIN
Trigger	On completion of every activity a beep has to be generated by the system
Scenario	1. Customer observes the control panel 2. Customer enters the ATM card 3. Customer enters the PIN 4. Customer selects the operation - withdrawal 5. Customer collects the cash, statement, card etc

Exceptions	1. The control panel is not ready 2. PIN is incorrect 3. Card is not recognised 4. Insufficient balance 5. Limit for the transaction exceeds 6. Total number of allowed transactions per day
Priority	Essential and must be implemented in banking system
Secondary Actor	Administrator
Open Issues	1. Is there a need to display any additional information by the control panel ? 2. For how many time the PIN is allowed to enter on incorrect supplement. 3. How much time is allotted to the customer to pick the items like cash or card after completion of operation ?

Example 2.8.3 *Develope complete use case for the following : Searching for books based on title in an on-line book store.*

Solution :

Use Case	On-line book store
Primary Actor	Customer
Goal in Context	To monitor all the functionalities required to establish the session

Scenario	1. Customer logs in to purchase the book online 2. Customer browses through the catalog based on title of the book 3. Customer selects the book and adds it to the shopping cart. 4. The contents of the shopping cart are displayed. 5. The book details and shipping information is confirmed by the customer. 6. The customer pays for the book
Extensions	6a The customer pays by credit card (include relationship) 6b The customer pays by the cheque (include relationship) 6c The customer pays by cash (include relationship)

Example 2.8.4 *A coffee vending machine dispenses coffee to customers. Customers order coffee by selecting a recipe from a set of recipes. Customers pay for the coffee using coins. Change is given back, if any, to the customers. The 'Service Assistant' loads ingredients (coffee. powder, milk, sugar, water, chocolate) into the coffee machine. The 'Service Assistant' adds a recipe by indicating the name of the coffee, the units of coffee powder, milk, sugar, water and chocolate to be added as well as the cost of the coffee. The Service Assistant can also edit and delete a recipe. i) Develop the use case diagram for the specification above. (Marks 6) ii) For any two scenarios draw an activity diagram and sequence diagram. (5+5 Marks)*

AU : Dec.-13, Marks 16

Solution : **i)** The use case diagram is as given below -

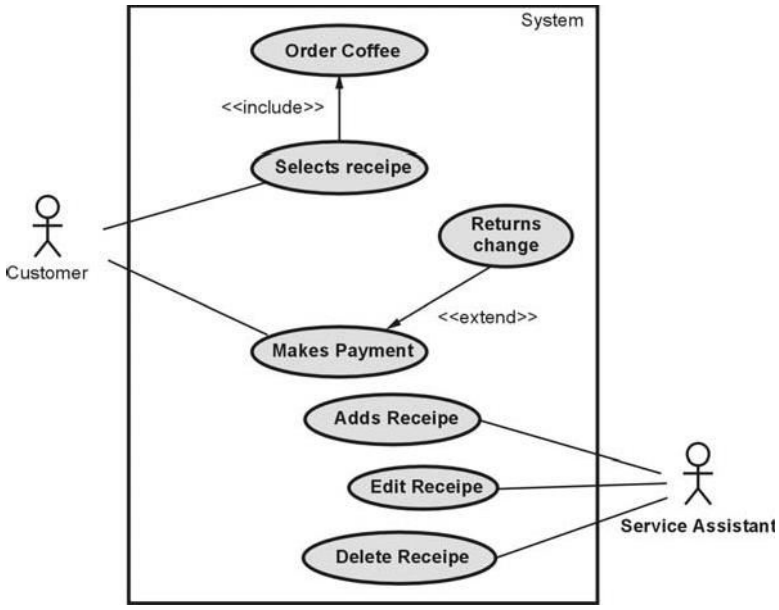


Fig. 2.8.9

ii) Activity Diagram

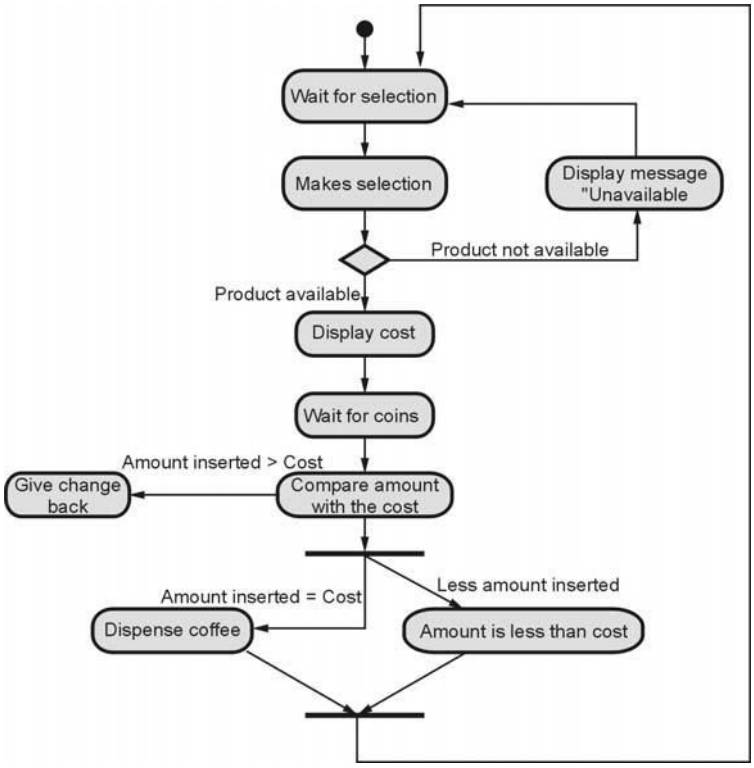


Fig. 2.8.10 Activity diagram for customer orders for the coffee

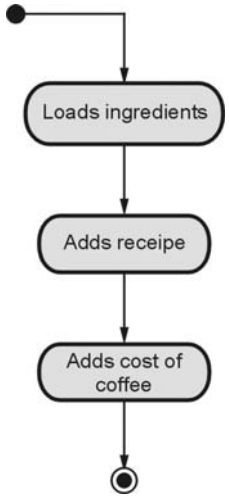


Fig. 2.8.11 Activity diagram for service assistant for adding the receipe

Sequence Diagram :

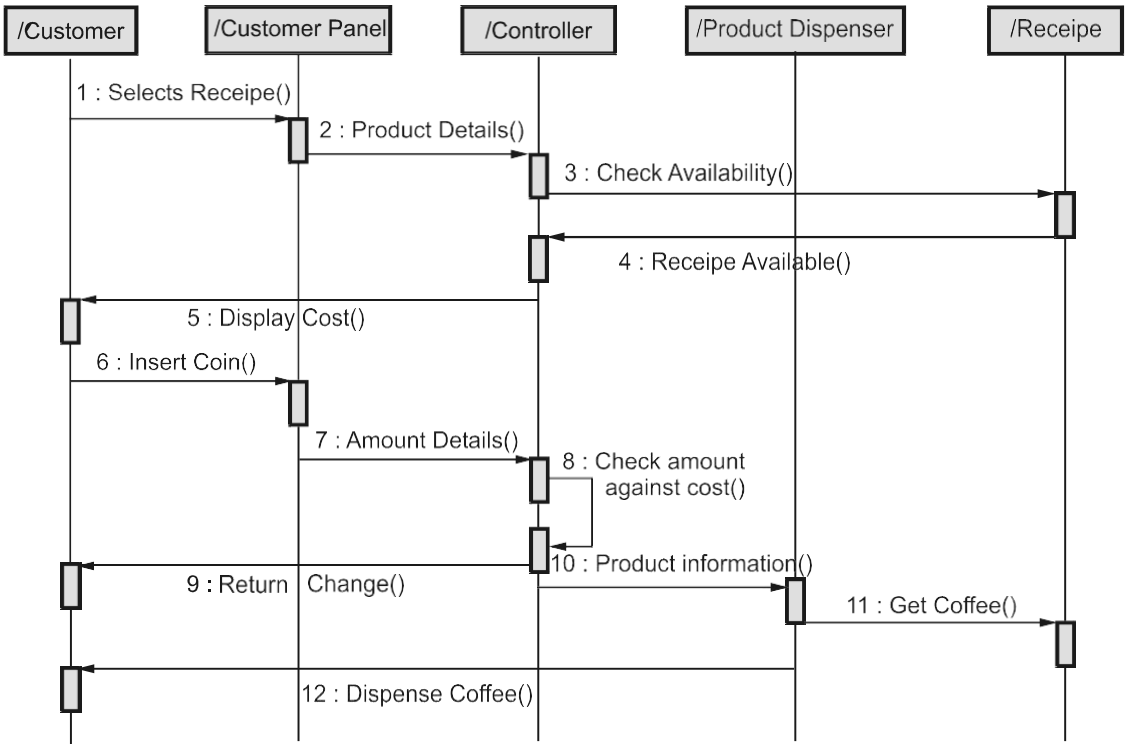


Fig. 2.8.12 Customer getting coffee from vending machine

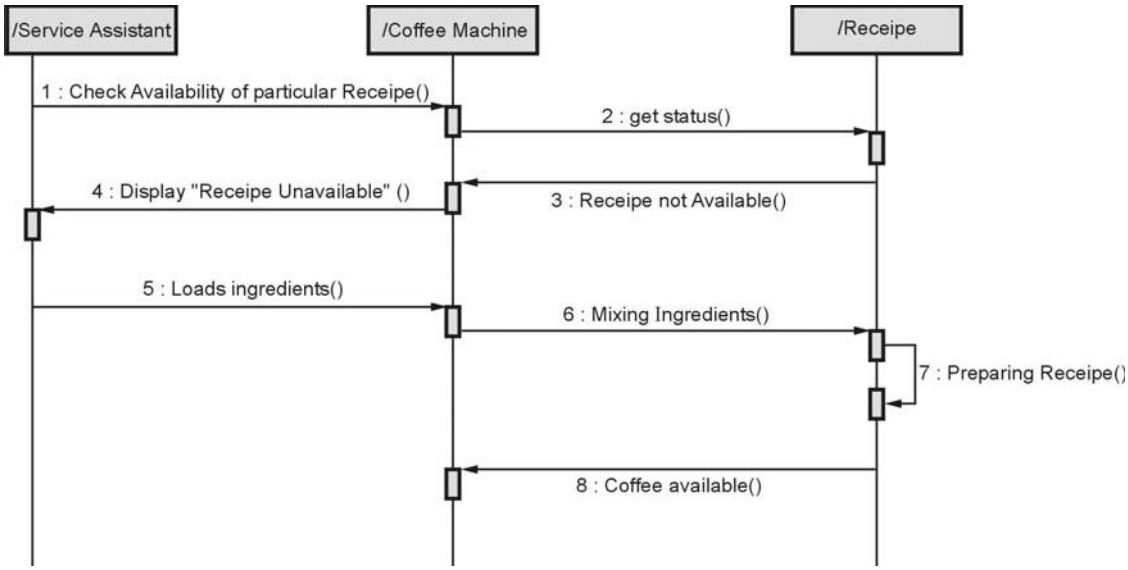


Fig. 2.8.13 Service assistant loads the ingredient and prepare coffee

Requirement Gathering

Requirements Gathering is the systematic process of identifying, collecting, analyzing, and documenting the needs and expectations of stakeholders for a software system.

It is one of the most critical phases of the **Software Development Life Cycle (SDLC)** because the success of the project largely depends on how well the requirements are understood.

Objectives of Requirements Gathering

- Understand **what the customer actually needs**
- Define **system scope and boundaries**
- Reduce ambiguity, risks, and rework
- Serve as a foundation for **design, development, testing, and maintenance**

Types of Requirements

1. Functional Requirements

Describe **what the system should do**.

- User login and authentication
- Report generation
- Data processing rules

2. Non-Functional Requirements

Describe **how the system should perform**.

- Performance (response time, throughput)
- Security
- Scalability
- Reliability
- Usability

3. Business Requirements

High-level objectives of the organization.

- Improve customer satisfaction
- Reduce operational costs

4. User Requirements

End-user expectations.

- User-friendly interface
- Easy navigation

Stakeholders Involved

- Clients / Customers
- End users
- Project managers
- Business analysts
- Developers
- Testers
- Domain experts

Requirements Gathering Techniques

1. Interviews

- One-to-one or group discussions
- Useful for detailed understanding

2. Questionnaires / Surveys

- Cost-effective for large user groups
- Structured responses

3. Workshops (JAD – Joint Application Development)

- Collaborative sessions
- Faster consensus

4. Observation

- Watching users perform tasks
- Useful when users cannot clearly articulate needs

5. Document Analysis

- Study existing systems, manuals, policies

6. Prototyping

- Early working models
- Helps clarify unclear requirements

7. Brainstorming

- Generate innovative ideas
- Encourages stakeholder participation

Steps in Requirements Gathering Process

1. **Identify stakeholders**
2. **Elicit requirements** using appropriate techniques
3. **Analyze requirements** for feasibility and conflicts
4. **Prioritize requirements**
5. **Document requirements** (SRS – Software Requirements Specification)
6. **Validate requirements** with stakeholders
7. **Manage changes** (requirements management)

Challenges in Requirements Gathering

- Ambiguous or incomplete requirements
- Changing customer needs
- Communication gaps
- Stakeholder conflicts
- Lack of domain knowledge

Output of Requirements Gathering

- **Software Requirements Specification (SRS)**
- Use case diagrams
- User stories (Agile)
- Requirement Traceability Matrix (RTM)

Developing Use Cases

Use Cases describe how users (actors) interact with a system to achieve specific goals. They are widely used during the **requirements analysis phase** to clearly capture functional requirements in a user-centric manner.

What is a Use Case

A **Use Case** is a structured description of a sequence of actions performed by a system in response to an actor to achieve a goal.

📌 *Focus:* **What the system does**, not how it is implemented.

Components of a Use Case

1. Actor

- A person, system, or external entity interacting with the system
- Example: Student, Admin, Bank Server

2. Use Case

- A specific functionality of the system
- Example: Login, Register Course, Make Payment

3. System Boundary

- Defines the scope of the system

Steps for Developing Use Cases

Step 1: Identify Actors

Ask:

- Who uses the system?
- Who provides input?
- Who receives output?
- Who maintains the system?

Step 2: Identify Actor Goals

- Determine what each actor wants to achieve

- Each goal usually becomes one use case

Step 3: Define Use Cases

- Name use cases using **verb + noun**
    - Example: *Submit Application*, *Generate Report*

Step 4: Draw Use Case Diagram

- Use UML notation:
    - Stick figure → Actor
    - Oval → Use Case
    - Rectangle → System boundary

Step 5: Describe Use Case Scenarios

Write detailed descriptions including:

- Main flow (happy path)
- Alternate flows
- Exception flows

Step 6: Validate with Stakeholders

- Ensure completeness and correctness
- Remove ambiguity

Example Use Case (Student Management System)

Use Case Name: Register Course
Actor: Student

Precondition: Student is logged in
Postcondition: Course is successfully registered

Main Flow:

1. Student selects course registration
2. System displays available courses
3. Student chooses a course
4. System validates eligibility
5. System confirms registration

Alternate Flow:

- If eligibility criteria not met → display error message

Use Case Relationships

1. «include»

- Mandatory reuse of functionality
- Example: *Place Order* «include» *Make Payment*

2. «extend»

- Optional or conditional behavior
- Example: *Login* «extend» *Forgot Password*

3. Generalization

- Inheritance between use cases or actors

Benefits of Use Cases

- Clear understanding of system behavior
- Easy communication with stakeholders
- Helps in test case generation
- Supports requirement traceability

Use Case template	
Use Case	Write the name of the scenario.
Primary Actor	Specify the role of the entity who interacts with the system.
Goal in Context	Specify the purpose of this use case scenario.
Preconditions	The condition that is to be satisfied before the use case start.
Scenario	Sequence of steps that describe the main scenario.
Triggers **Extensions points**	The triggering event due to which the function starts, is described. The extension points describe the extend relationship.
Exceptions	Describe the exceptional situations that may occur during the execution of the use case.
Priority	Describe the necessity of the system to be implemented.
Open Issues	Certain additional issues that are required for execution of the scenario and that are not specified in the main scenario.

Write a use case modeling for ATM System

Use Case template	
Use Case	ATM system
Primary Actor	Customer
Goal in Context	To monitor all the functionalities required to establish the session
Preconditions	ATM system has to be programmed and as to recognise and validate the PIN.
Trigger	On completion of every activity a beep has to be generated by the system.
Scenario	1. Customer observes the control panel. 2. Customer enters the ATM card 3. Customer enters the PIN. 4. Customer selects the operation(withdrawal, deposit, inquiry) 5. Customer collects the cash, statement, card etc.
Exceptions	1. The control panel is not ready. 2. PIN is incorrect. 3. Card is not recognised. 4. Insufficient balance 5. Limit for the transaction exceeds. 6. Total number of allowed transactions per day.
Priority	Essential and must be implemented in banking system.
Secondary Actor	Administrator
Open Issues	1. Is there a need to display any additional information by the control panel ? 2. For how many time the PIN is allowed to enter on incorrect supplement. 3. How much time is allotted to the customer to pick the items like cash or card after completion of operation ?

Negotiating and Validating Requirements

Negotiating and Validating Requirements are crucial activities in the **requirements engineering process** that ensure the final requirements are **feasible, agreed upon, correct, and complete** before development begins.

1. Negotiating Requirements

Requirements negotiation is the process of **resolving conflicts and prioritizing requirements** among stakeholders to reach a mutually acceptable agreement.

Different stakeholders often have:

- Conflicting goals
- Limited budgets and time
- Different priorities

Negotiation helps balance these constraints.

Why Requirements Negotiation is Needed

- Limited resources (time, cost, manpower)
- Conflicting stakeholder demands
- Technical feasibility issues
- Regulatory or policy constraints
- Risk management

Activities in Requirements Negotiation

1. Identify Conflicts

- Conflicts between functional vs non-functional requirements
- Business needs vs technical feasibility
- Cost vs quality

2. Prioritize Requirements

Common prioritization techniques:

- **MoSCoW Method**
  - Must have
  - Should have
  - Could have
  - Won’t have

- Value vs Risk analysis
- Cost–benefit analysis

3. Trade-off Analysis

- Decide what to include, modify, or drop
- Balance performance, security, cost, and schedule

4. Stakeholder Agreement

- Discuss alternatives
- Document accepted requirements
- Obtain formal approval (sign-off)

Techniques Used in Negotiation

- Stakeholder meetings and workshops
- Prototyping
- Scenarios and use cases
- Cost and impact analysis
- Win–win negotiation approach

2. Validating Requirements

Requirements validation ensures that:

- The documented requirements **accurately represent stakeholder needs**
- The requirements are **ready for implementation**

📌 *Key Question:*
"Are we building the right system?"

Objectives of Requirements Validation

- Ensure correctness and completeness
- Remove ambiguity and inconsistency
- Confirm feasibility
- Ensure testability and traceability

Validation Checklist

Requirements should be:

- Correct
- Complete
- Consistent
- Unambiguous
- Feasible
- Verifiable
- Traceable
- Prioritized

Techniques for Requirements Validation

1. Requirements Reviews

- Formal or informal peer reviews
- Walkthroughs and inspections

2. Prototyping

- Early system models to confirm understanding

3. Test Case Derivation

- Writing test cases from requirements
- Helps ensure clarity and testability

4. Stakeholder Validation Meetings

- Customer confirms and approves requirements

5. Use Case Validation

- Ensure all user interactions are covered

3. Difference Between Negotiation and Validation

Aspect	Negotiation	Validation
Purpose	Resolve conflicts & prioritize	Confirm correctness
Focus	Agreement	Accuracy & quality
Participants	Stakeholders, PM, Analysts	Stakeholders, QA, Analysts
Outcome	Agreed requirements	Approved requirements

4. Outputs

- Finalized and prioritized requirements list
- Approved **Software Requirements Specification (SRS)**
- Requirement Traceability Matrix (RTM)
- Stakeholder sign-off document

5. Importance

- Prevents scope creep
- Reduces development risks
- Saves cost and time
- Improves stakeholder satisfaction
- Ensures successful project delivery

Unified Modeling Language (UML)

Unified Modeling Language (UML) is a **standard visual modeling language** used in software engineering to **specify, visualize, construct, and document** the artifacts of a software system. It provides a common language for developers, designers, analysts, and stakeholders.

Definition

UML is a standardized modeling language developed by **Grady Booch, Ivar Jacobson, and James Rumbaugh** and maintained by the **Object Management Group (OMG)**.

Objectives of UML

- Visualize system architecture and design
- Specify system structure and behavior
- Document software systems
- Improve communication among stakeholders
- Support object-oriented analysis and design (OOAD)

Key Building Blocks of UML

1. Things

- **Structural things:** Class, Interface, Component, Node
- **Behavioral things:** Interaction, State machine
- **Grouping things:** Package
- **Annotational things:** Note

2. Relationships

- **Association**
- **Aggregation**
- **Composition**
- **Generalization (Inheritance)**
- **Dependency**
- **Realization**

3. Diagrams

UML consists of **14 standard diagrams**, grouped into **Structural** and **Behavioral** diagrams.

UML Diagram Classification

A. Structural Diagrams

Describe the **static structure** of the system.

1. **Class Diagram**
    - Shows classes, attributes, methods, and relationships
2. **Object Diagram**
    - Snapshot of objects at a particular time
3. **Package Diagram**
    - Organizes model elements into packages
4. **Component Diagram**
    - Shows software components and dependencies
5. **Deployment Diagram**
    - Shows hardware and software deployment
6. **Composite Structure Diagram**
7. **Profile Diagram**

B. Behavioral Diagrams

Describe the **dynamic behavior** of the system.

1. Use Case Diagram

- Shows interactions between actors and the system
- Captures functional requirements

2. Activity Diagram

- Represents workflows and business processes

3. State Machine Diagram

- Shows state transitions of an object

4. Interaction Diagrams

(Subset of behavioral diagrams)

- **Sequence Diagram**
- **Communication Diagram**
- **Timing Diagram**
- **Interaction Overview Diagram**

UML in Software Development Phases

SDLC Phase	UML Usage
Requirements	Use Case, Activity
Design	Class, Sequence
Implementation	Component
Deployment	Deployment Diagram
Testing	Sequence, State

Advantages of UML

- Standard and widely accepted
- Improves design quality
- Easy to understand visual representation
- Supports complex system modeling
- Technology independent

Limitations of UML

- Time-consuming for small projects
- Requires modeling expertise
- Does not enforce methodology
- Can become complex if overused

Static and Dynamic Models in (UML)

In **software modeling**, systems are represented using **static** and **dynamic** models to understand both the **structure** and **behavior** of a system. These models are commonly expressed using **UML diagrams**.

1. Static Model

A **Static Model** represents the **structural aspects** of a system. It shows **what the system is made of**, not how it behaves over time.

📌 *Focus:* **System structure**

Characteristics

- Describes system components
- Shows relationships among classes or objects
- Independent of time
- Does not show interactions or workflows

UML Diagrams Used for Static Modeling

- **Class Diagram**
- **Object Diagram**
- **Package Diagram**
- **Component Diagram**
- **Deployment Diagram**

Example (Static Model – Class Diagram)

A *Student Management System* showing:

- Classes: Student, Course, Faculty
- Attributes: name, rollNo
- Relationships: association, inheritance

Advantages of Static Models

- Clear understanding of system architecture
- Helps in database and code design
- Easy to maintain and extend

2. Dynamic Model

A **Dynamic Model** represents the **behavioral aspects** of a system. It shows **how the system behaves over time** and how components interact.

📌 *Focus:* **System behavior and interactions**

Characteristics

- Time-dependent
- Shows flow of control and data
- Represents system reactions to events
- Captures business logic and workflows

UML Diagrams Used for Dynamic Modeling

- **Use Case Diagram**
- **Sequence Diagram**
- **Activity Diagram**
- **State Machine Diagram**
- **Communication Diagram**
- **Timing Diagram**

Example (Dynamic Model – Sequence Diagram)

A *Login Process* showing:

1. User enters credentials
2. System validates user
3. Access granted or denied

Advantages of Dynamic Models

- Clarifies system behavior
- Helps in understanding complex logic
- Useful for test case generation
- Identifies missing requirements

3. Difference Between Static and Dynamic Models

Aspect	Static Model	Dynamic Model
Focus	Structure	Behavior
Time Dependency	No	Yes
Shows	Classes, relationships	Interactions, workflows
UML Diagrams	Class, Object, Component	Use Case, Sequence, Activity
Used For	System design	Requirement analysis & testing

Use-Case Diagram (UML)

A **Use-Case Diagram** is a **behavioral UML diagram** that represents the **functional requirements** of a system by showing the **interactions between actors and the system**.

📌 *Focus:* **What the system does**, not how it is implemented.

Definition

A Use-Case Diagram shows the interactions between external actors and a system to achieve specific goals.

Purpose of Use-Case Diagram

- Capture **functional requirements**
- Identify system boundaries
- Understand user interactions
- Communicate system behavior to stakeholders
- Serve as a base for test cases

Main Components

1. Actor

- An external entity interacting with the system
- Can be a **person, device, or another system**
- Represented by a **stick figure**

Examples: Student, Admin, Payment Gateway

2. Use Case

- A functionality provided by the system
- Represented by an **oval**
- Named using **verb + noun**

Examples: Login, Register Course, Make Payment

3. System Boundary

- Represents the scope of the system
- Shown as a **rectangle**

4. Relationships

a) Association

- Communication link between actor and use case

b) «include»

- Mandatory behavior reused by another use case
- Example: *Place Order* «include» *Make Payment*

c) «extend»

- Optional or conditional behavior
- Example: *Login* «extend» *Forgot Password*

d) Generalization

- Inheritance between actors or use cases

Steps to Draw a Use-Case Diagram

1. Identify system scope
2. Identify actors
3. Identify use cases for each actor
4. Draw system boundary
5. Connect actors to use cases
6. Apply include, extend, generalization if required

Example: Online Shopping System

Actors:

- Customer
- Admin
- Payment Gateway

Use Cases:

- Register
- Login
- Browse Products
- Place Order
- Make Payment
- Manage Products

Advantages

- Easy to understand
- Improves communication
- Helps in requirement validation
- Supports test case generation

Limitations

- Does not show internal logic
- Not suitable for complex workflows
- Lacks timing information

Design a Use case diagram for Library Management Systems

:

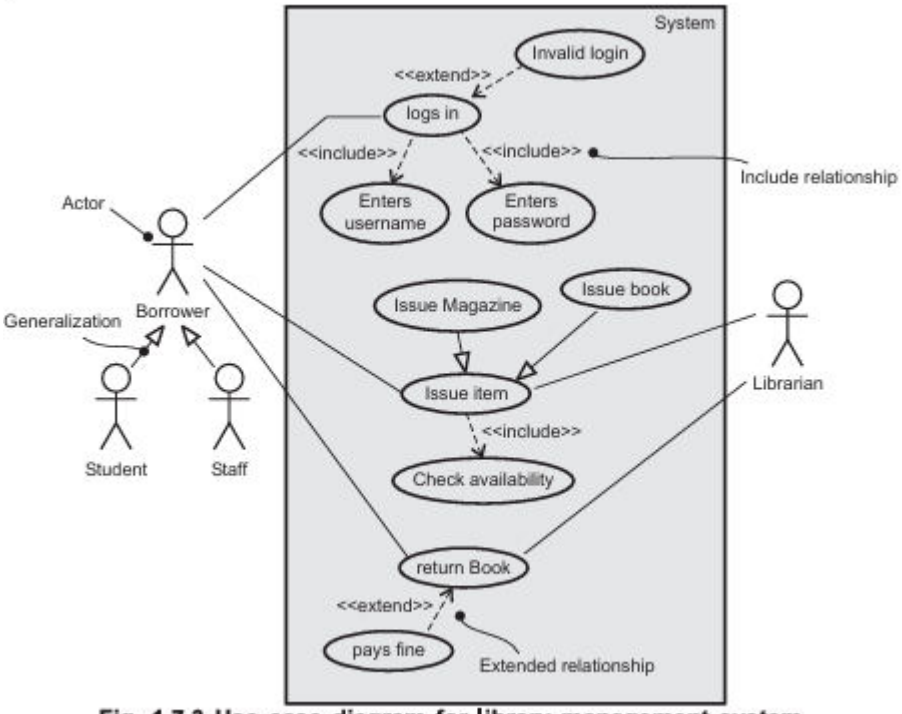
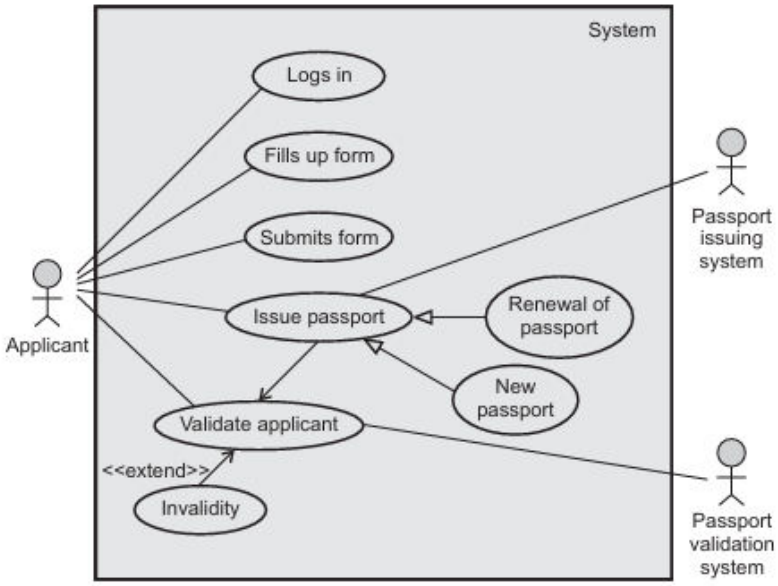


Fig. 1.7.3 Use case diagram for library management system

Design a Use diagram for Pass port Automation



UML Class Diagram

A **UML Class Diagram** is a **structural (static) diagram** that represents the **structure of a system** by showing its **classes, attributes, methods, and relationships** among classes.

📌 *Focus:* **System structure and object-oriented design**

A Class Diagram describes the static structure of a system by modeling classes, their attributes, operations, and relationships.

Purpose of Class Diagram

- Visualize system architecture
- Support object-oriented analysis and design
- Serve as a blueprint for coding
- Help in database design
- Improve maintainability

Components of a Class Diagram

1. Class

Represented as a **rectangle divided into three sections**:

```
------------------
| Class Name     |
------------------
| Attributes     |
------------------
| Methods        |
------------------
```

2. Attributes

- Data members of a class
- Format:
  visibility name : datatype

Visibility symbols:

- + Public
- - Private
- # Protected

Example:
```
- rollNo : int
```

3. Methods (Operations)

- Functions performed by the class
- Format:
  visibility methodName(parameters) : returnType

Example:
```
+ calculateGPA() : float
```

Relationships in Class Diagram

1. Association

- Represents a **uses-a** relationship
- Can be uni-directional or bi-directional

2. Aggregation

- **Whole–part** relationship (weak)
- Represented by a **hollow diamond**
- Example: *Department* has *Faculty*

3. Composition

- **Strong whole–part** relationship
- Represented by a **filled diamond**
- Part cannot exist without whole
- Example: *Order* contains *OrderItem*

Difference between aggregation and composition

Aggregation	Composition
The relationship between child and parent class is represented in aggregation relationship where in child can exist independently of the parent.	The relationship between child and parent class is represented in composition relationship where in child can not exist independently of the parent.
It is weak association.	It is strong association.
Deletion of owning class doesn't affect system.	If owning class is deleted it will significantly affect the system.
The hollow diamond is used in arrow notation.	The solid diamond is used in arrow notation.
For example	For example

4. Generalization (Inheritance)

- **Is-a** relationship
- Represented by a **hollow triangle**
- Example: *Student* ← *UGStudent*

5. Dependency

- Temporary relationship
- Represented by a **dashed arrow**

6. Realization

- Used for interfaces
- Represented by a **dashed line with hollow triangle**

Example: Student Management System

Classes:

- Student
- Course
- Faculty

Relationships:

- Student *registers* Course (Association)
- Course *taught by* Faculty (Association)

Advantages of Class Diagram

- Clear visualization of system design
- Supports code generation
- Encourages reuse
- Easy to modify and extend

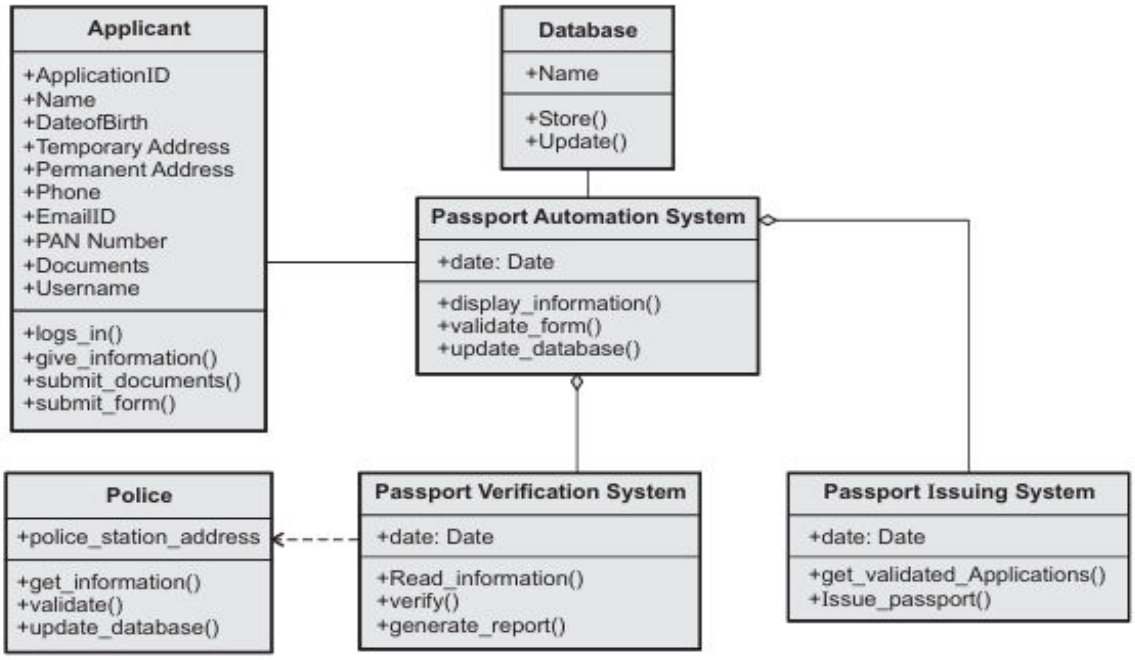
Limitations

- Does not represent dynamic behavior
- Complex for large systems if not modularized

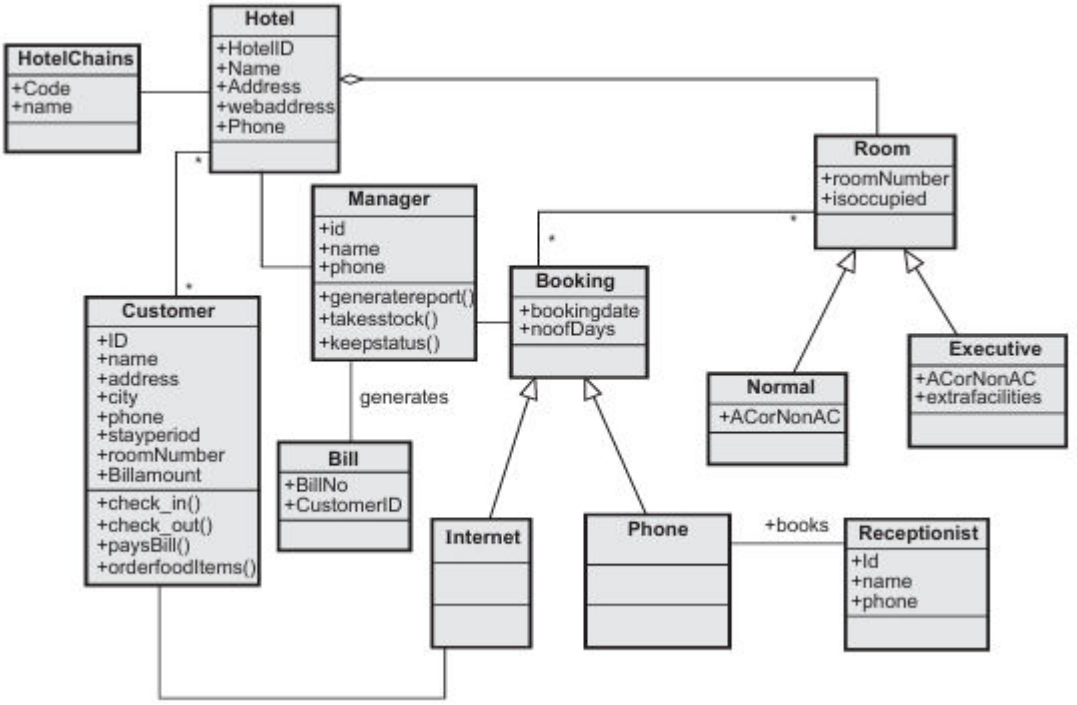
WHEN TO USE CLASS DIAGRAM

1. For modeling the vocabulary of the system the class diagram is used.
2. For modeling the collaborations of the system the class diagram is used.
3. For modeling the logical database schema the class diagram is used.

Design a Class diagram for passport Automation



Example 2.16.2 *Prepare a class model for the hotel management system. The system should support chain of hotels. A hotel contains two categories of rooms : executive and normal, both AC and non-AC. The customers of executive rooms can avail extra facilities like games, swimming, food service in rooms, etc. The booking is possible by Internet or by phone. If the booking is through phone, process is done by receptionist and if booking is done through Internet the process is carried out by customer through hotel website. Depending on the number of days customer stays, appropriate bill is generated. The bill also contains amount for transport, food and other facilities enjoyed by the customer along with necessary taxes. The manager should be able to generate reports like list of customers staying in the hotel, list of rooms empty, monthly/yearly income, etc.*



Example 2.16.4 *Convert the following into a class diagram with appropriate classes, relationships, multiplicity.*

Car dealer sales cars. A car is owned by an owner. The owner has an address. The owner can be a person, a company or a bank. A car loan may be involved in the purchase of a car. Bank provides the loan.

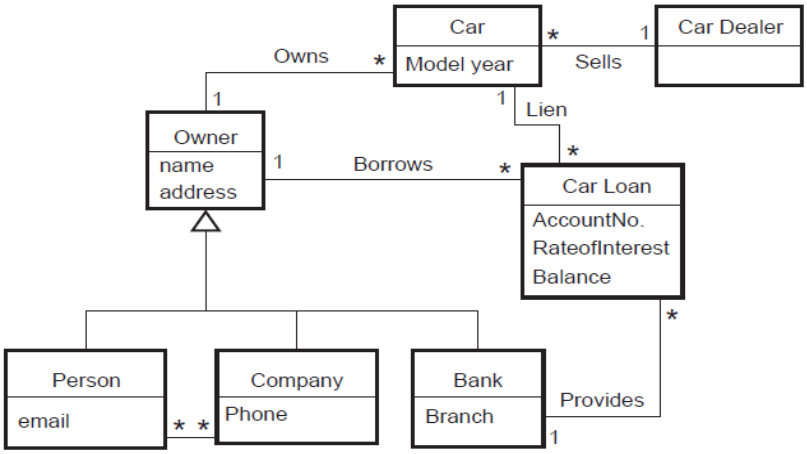


Fig. 2.16.2 Class diagram

Example 2.16.7 *A project has three to five students. A projects has a guide can guide one to three projects. For this description draw a class diagram. From the class diagram draw an object to show two projects, seven students and one guide. Do not write any explanation, just draw the diagrams.*

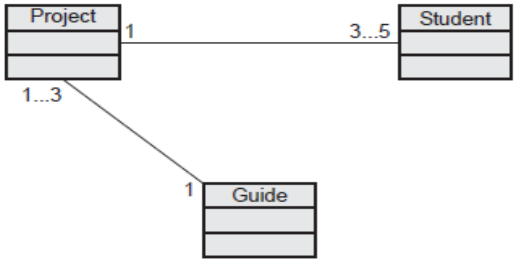
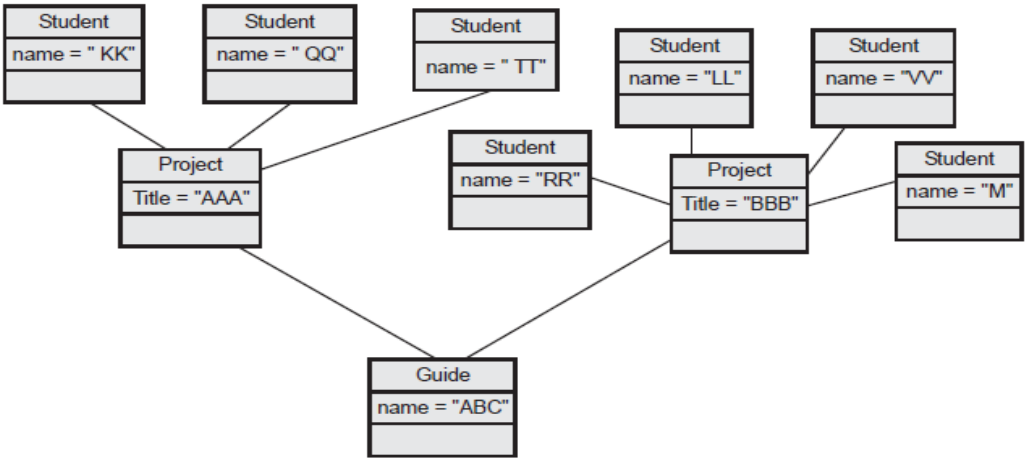
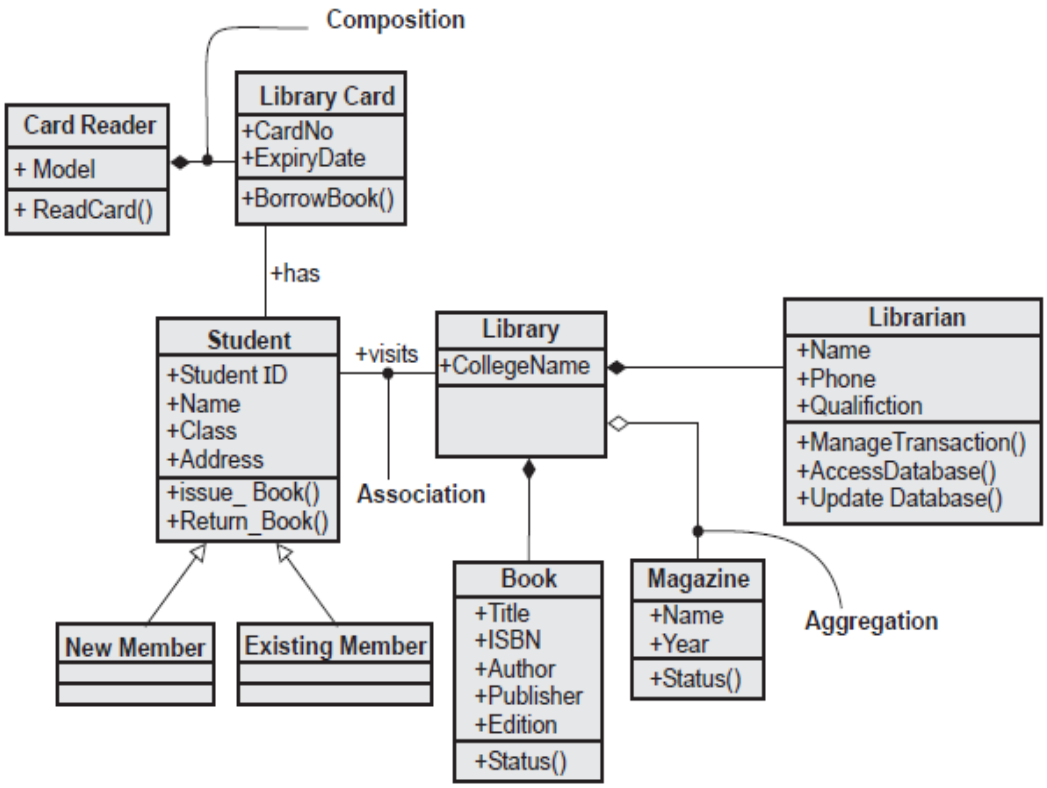


Fig. 2.16.5 Class diagram



Example 2.16.14 *Construct design for Library Information system which comprises and following notations (i) Aggregation (ii) Composition (iii) Association* **AU : Dec.-15, Marks 16**

Solution :



Example 2.16.15 *Model a class diagram for "Banking System" State the functional requirements you are considering.* **AU : Dec.-17,18, Marks 15; Dec.-18, Marks 13**

Solution : See Fig. 2.16.10 on next page.

Functional Requirements -

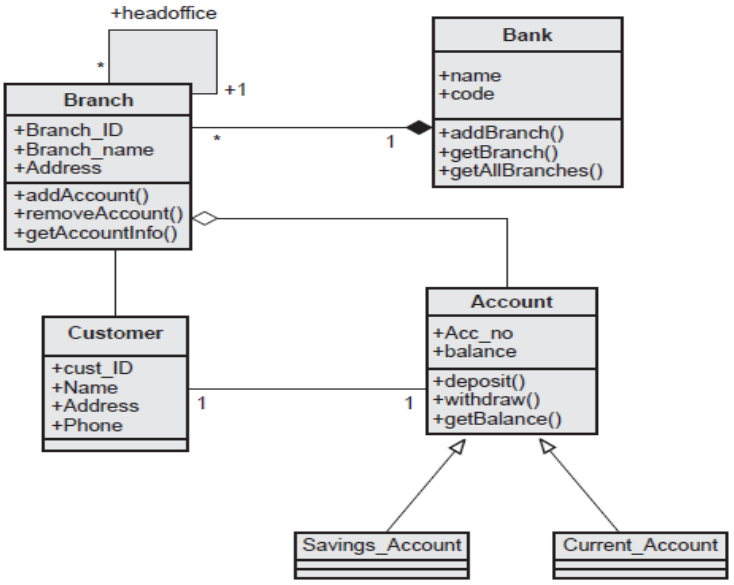
1) The valid customer must log in to the system using username and password.

2) The system should authenticate the user.

3) The system should allow the transfer the funds between authorised access.

4) The system should allow the customer to view the balance in his account.

5) An administrator is that person who is allowed for some editing for the internet banking system like add/cancel customer, check the transactions.



Activity Diagram (UML)

An **Activity Diagram** is a **behavioral UML diagram** that represents the **workflow of activities** in a system. It shows the **flow of control or data** from one activity to another.

📌 *Focus:* **Process flow and business logic**

An Activity Diagram models the dynamic aspects of a system by showing the sequence of activities and decisions involved in a process.

Purpose of Activity Diagram

- Model business processes
- Represent workflows
- Describe use case behavior
- Identify parallel activities
- Support requirement analysis

Key Elements of Activity Diagram

1. Initial Node

- Start of the activity
- Represented by a **filled black circle**

2. Activity / Action

- A task or operation
- Represented by a **rounded rectangle**

3. Control Flow

- Arrow showing the sequence of activities

4. Decision Node

- Represents a condition (Yes/No)
- Shown as a **diamond**

5. Merge Node

- Combines alternative flows

6. Fork Node

- Splits flow into parallel activities

7. Join Node

- Synchronizes parallel activities

8. Swimlanes

- Partition activities based on responsibility
- Shows who does what

9. Final Node

- End of the activity
- Represented by a **filled circle inside a hollow circle**

Steps to Draw an Activity Diagram

1. Identify the process or use case
2. List all activities involved
3. Identify decisions and conditions
4. Arrange activities in sequence
5. Add parallel flows if required
6. Mark start and end points

Example: Online Exam System (Activity Flow)

1. Start
2. Student logs in
3. System validates credentials
4. [Valid?]
    - Yes → Display exam
    - No → Show error
5. Student answers questions
6. Submit exam
7. End

Advantages

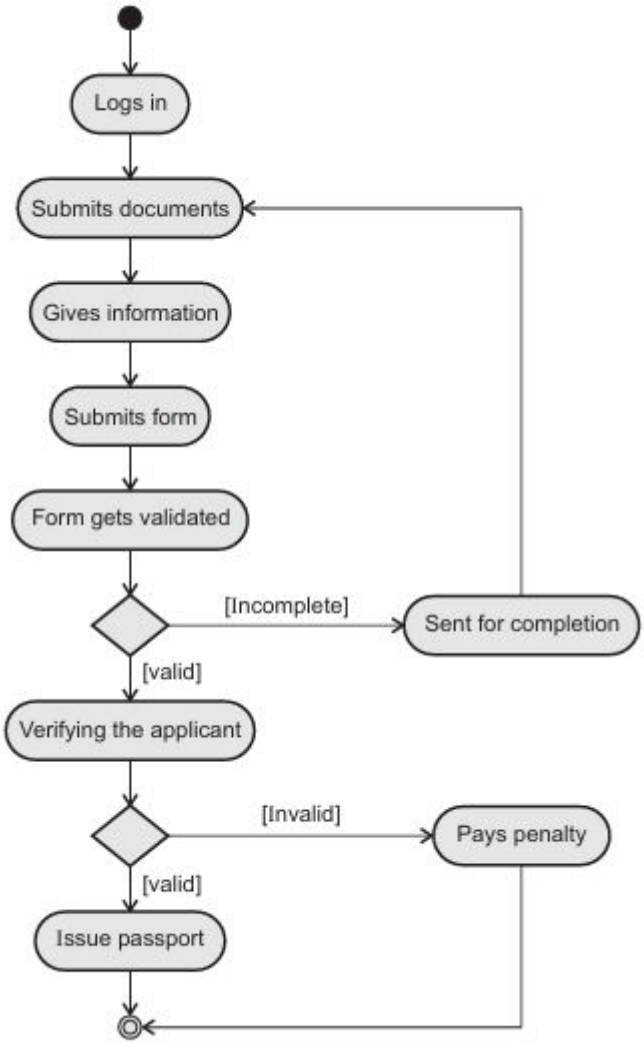
- Easy to understand
- Clearly represents workflows

- Supports parallel processing
- Useful for both technical and non-technical users

Limitations

- Not suitable for detailed object interactions
- Can become complex for large systems

Design a Activity diagram for Pass port Automation



Sequence Diagram (UML)

A **Sequence Diagram** is an **interaction (behavioral) UML diagram** that shows **how objects interact with each other over time** to carry out a specific functionality.

📌 *Focus:* **Order of message exchanges (time sequence)**

Definition

A Sequence Diagram represents interactions among objects in a time-ordered sequence of messages.

Purpose of Sequence Diagram

- Understand object interactions
- Model dynamic behavior of a system
- Validate use case logic
- Identify responsibilities of objects
- Support test case design

Key Elements of a Sequence Diagram

1. Actor

- External entity initiating interaction
- Represented by a **stick figure**

2. Lifeline

- Represents the existence of an object over time
- Shown as a **vertical dashed line**

3. Object

- Participant in interaction
- Represented as a **rectangle on top of lifeline**

4. Activation Bar

- Shows period when an object is active
- Represented by a **thin vertical rectangle**

5. Message

- Communication between objects
- Shown as **arrows**

Types of messages:

- Synchronous (solid arrow)
- Asynchronous (open arrow)
- Return message (dashed arrow)
- Self-message (loop back)

6. Guard Condition

- Condition written in **[ ]**
- Controls message flow

Steps to Draw a Sequence Diagram

1. Identify the use case
2. Identify actors and objects
3. Arrange objects horizontally
4. Draw lifelines
5. Add messages in time order (top to bottom)
6. Add conditions and loops if needed

Example: Login Process

Objects:

- User
- Login Page
- Authentication Server
- Database

Flow:

1. User enters credentials
2. Login Page sends request
3. Server validates with Database
4. Result returned
5. User granted or denied access

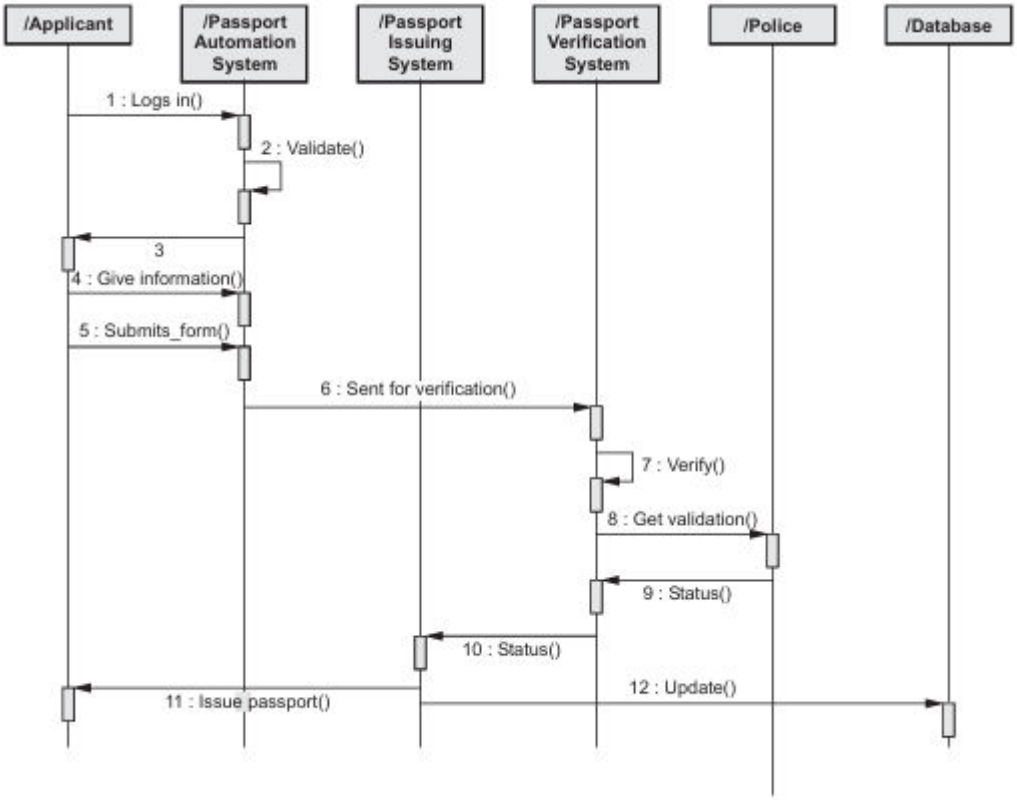
Advantages

- Clearly shows message sequence
- Easy to understand interactions
- Helps in debugging design issues
- Useful for complex logic clarification

Limitations

- Not suitable for very large systems
- Does not show object relationships

Design Sequence Diagram for Pass Port Automation System



Package Diagram (UML)

A **Package Diagram** is a **structural UML diagram** used to organize and group related model elements into **packages** and to show **dependencies between those packages**.

📌 *Focus:* **High-level system organization**

Definition

A Package Diagram represents the organization of a system into packages and the dependencies among them.

Purpose of Package Diagram

- Organize large systems into manageable units
- Show module-level architecture
- Improve understandability and maintenance
- Represent layered architecture
- Support reuse

Key Elements of a Package Diagram

1. Package

- A container that groups related elements (classes, interfaces, components)
- Represented as a **folder-like symbol**

Example:
`UI`, `Business Logic`, `Database`

2. Package Contents

- Classes, interfaces, sub-packages
- Shown inside the package (optional)

3. Dependency

- Indicates one package depends on another
- Represented by a **dashed arrow**

Meaning: Changes in the supplier package may affect the client package

4. Visibility

- Public (+)
- Private (-)

Steps to Draw a Package Diagram

1. Identify major system modules
2. Group related classes into packages
3. Name packages clearly
4. Draw dependencies between packages
5. Arrange packages logically (layered structure)

Example: Online Shopping System

Packages:

- `User Interface`
- `Order Management`
- `Payment Processing`
- `Database`

Dependencies:

- UI → Order Management
- Order Management → Payment Processing
- Payment Processing → Database

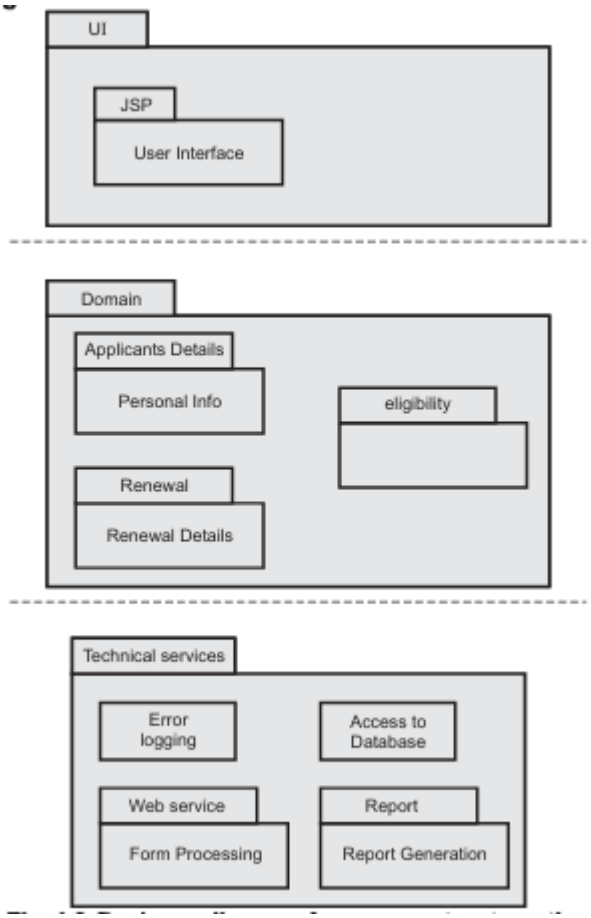
Advantages

- Simplifies large system design
- Shows high-level architecture
- Helps in team-based development
- Enhances modularity

Limitations

- Does not show internal class details
- No dynamic behavior representation

Design a Package diagram for Passport Automation



State Chart Diagram (UML)

A **State Chart Diagram** (also called **State Machine Diagram**) is a **behavioral UML diagram** that represents the **states of an object** and the **transitions between those states** in response to events.

📌 *Focus:* **State-dependent behavior of an object**

Definition

A State Chart Diagram models the dynamic behavior of an object by showing its states, transitions, and events.

Purpose of State Chart Diagram

- Describe object life cycle
- Model event-driven systems
- Show how an object reacts to events
- Useful for real-time and embedded systems
- Clarify complex conditional logic

Key Elements of a State Chart Diagram

1. State

- A condition or situation during the life of an object
- Represented by a **rounded rectangle**

Example: Idle, Processing, Completed

2. Initial State

- Starting point of the object
- Represented by a **filled black circle**

3. Final State

- End of object life cycle
- Represented by a **filled circle inside a hollow circle**

4. Transition

- Movement from one state to another
- Represented by an **arrow**

Label format:
```
event [condition] / action
```

5. Event

- Trigger that causes state change
  Example: submit(), cancel()

6. Guard Condition

- Boolean condition written in **[ ]**

7. Action / Activity

- Operation performed during transition or state

8. Composite State (Optional)

- A state that contains substates

Steps to Draw a State Chart Diagram

1. Identify the object whose behavior is to be modeled
2. Identify all possible states of the object
3. Identify events causing state changes
4. Define transitions between states
5. Mark initial and final states
6. Add guard conditions and actions

Example: Order Processing System

States:

- New
- Confirmed
- Shipped
- Delivered
- Cancelled

Transitions:

- Place Order → New
- Confirm Payment → Confirmed
- Ship Order → Shipped
- Deliver Order → Delivered
- Cancel Order → Cancelled

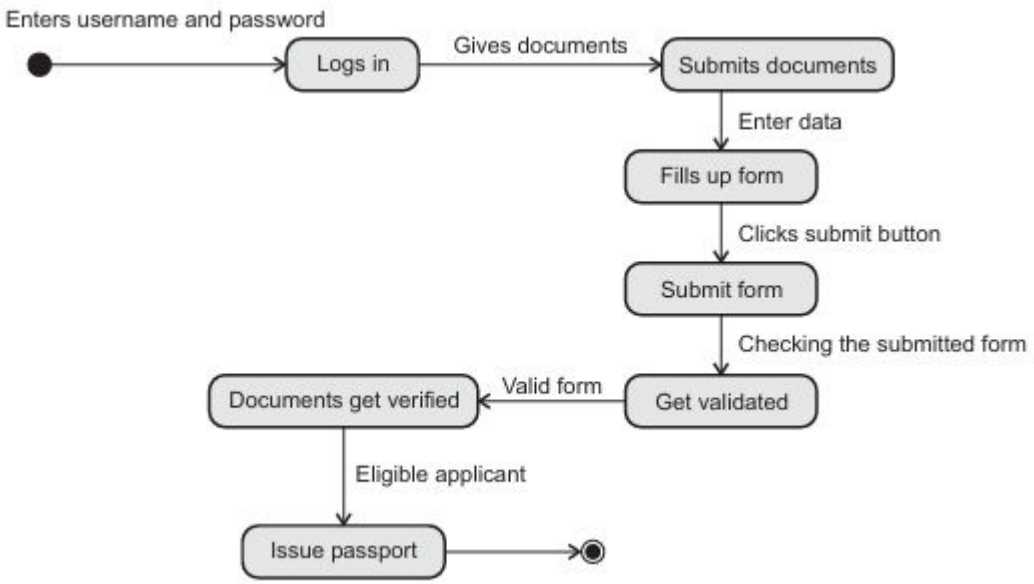
Advantages

- Clearly shows object behavior
- Easy to understand complex state logic
- Helpful in testing and validation
- Supports event-driven design

Limitations

- Not suitable for modeling overall system flow
- Can become complex for large systems

Design a State chart Diagram for pass port Automation



Component Diagram (UML)

A **Component Diagram** is a **structural UML diagram** that shows the **organization and dependencies of software components** in a system. It focuses on the **physical components** of the system and how they interact through interfaces.

📌 *Focus:* **Implementation-level system architecture**

A Component Diagram represents the physical components of a system and the dependencies among them.

Purpose of Component Diagram

- Model system architecture
- Show modularization of software
- Identify component dependencies
- Support reuse and maintainability
- Bridge design and implementation

Key Elements of a Component Diagram

1. Component

- A modular, replaceable part of a system
- Represented by a **rectangle with two small tabs** on the left side
- Examples: Authentication Module, Payment Module

2. Interface

- Defines services provided or required by a component

Types:

- **Provided Interface** – lollipop symbol
- **Required Interface** – socket symbol

3. Dependency

- Shows one component depends on another
- Represented by a **dashed arrow**

4. Ports (Optional)

- Interaction points of a component

Steps to Draw a Component Diagram

1. Identify major software components
2. Define provided and required interfaces
3. Draw components using standard notation
4. Connect interfaces and dependencies
5. Organize components logically

Example: Online Shopping System

Components:

- User Interface
- Order Management
- Payment Processing
- Inventory
- Database

Dependencies:

- UI → Order Management
- Order Management → Payment
- Payment → Bank Gateway
- Order Management → Inventory

Advantages

- Clear modular structure
- Supports parallel development
- Improves system maintainability
- Easy to understand architecture

Limitations

- Does not show internal component logic
- Not suitable for requirement analysis

Design a Component Diagram for Passport Automation

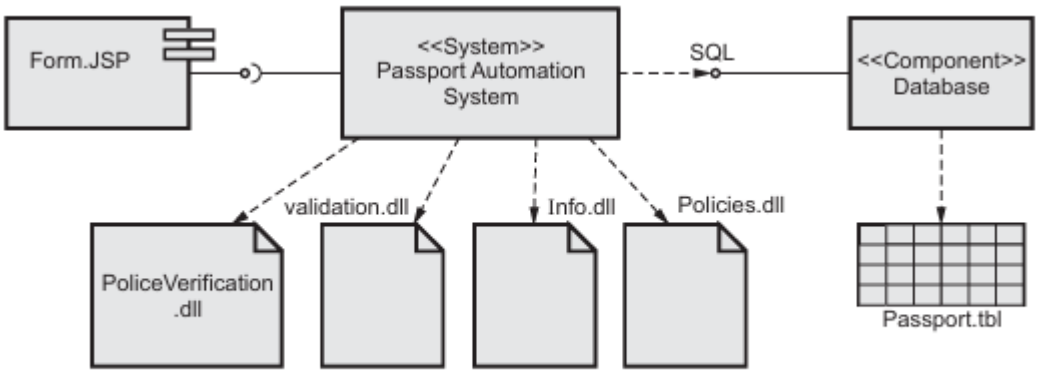


Fig. L3 Component diagram for passport automation system

Deployment Diagram (UML)

A **Deployment Diagram** is a **structural UML diagram** that represents the **physical deployment of software artifacts on hardware nodes**. It shows **where and how the system is deployed** in a real-world environment.

📌 *Focus:* **Physical architecture of the system**

A Deployment Diagram models the physical configuration of hardware nodes and the software components deployed on them.

Purpose of Deployment Diagram

- Show system's physical architecture
- Describe hardware–software mapping
- Support system installation and deployment
- Analyze performance, scalability, and security
- Useful in distributed systems

Key Elements of a Deployment Diagram

1. Node

- Physical or virtual hardware resource
- Represented by a **3D box**
- Examples: Server, Client PC, Mobile Device

2. Device

- A physical computing resource
- Example: Web Server, Database Server

3. Execution Environment

- Software environment running inside a node
- Example: JVM, Application Server

4. Artifact

- Deployable software elements
- Example: `.war`, `.exe`, `.jar`, database schema

5. Communication Path

- Connection between nodes
- Represents network links

Steps to Draw a Deployment Diagram

1. Identify hardware nodes
2. Identify execution environments
3. Identify deployable artifacts
4. Map artifacts to nodes
5. Draw communication paths

Example: Online Shopping System

Nodes:

- Client (Browser / Mobile)
- Web Server
- Application Server
- Database Server
- Payment Gateway Server

Artifacts:

- UI module
- Business logic module
- Database schema

Advantages

- Clear view of system deployment
- Helps in capacity planning
- Supports distributed system design
- Improves system reliability

Limitations

- Does not show internal software logic
- Requires detailed infrastructure knowledge

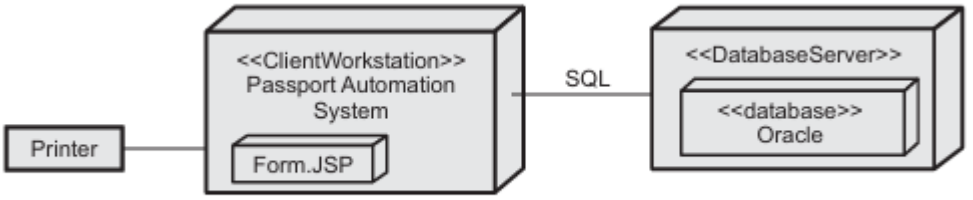


Fig. L4 Deployment diagram for passport automation system

Communication Diagram (UML)

A **Communication Diagram** (also called **Collaboration Diagram**) is an **interaction UML diagram** that shows **how objects communicate with each other** to perform a behavior. It emphasizes **object relationships** rather than the time sequence.

📌 *Focus:* **Object interactions and message flow**

Definition

A Communication Diagram models the interactions between objects in terms of messages and links, showing how they collaborate to achieve a use case or functionality.

Purpose of Communication Diagram

- Visualize object interactions
- Emphasize structural organization of objects
- Show message passing and sequence
- Support dynamic behavior modeling
- Useful for analyzing responsibilities

Key Elements of Communication Diagram

1. Objects

- Instances participating in interaction
- Represented as **rectangles with object name and class**
  Example: `:Customer`, `:OrderProcessor`

2. Links

- Connection between objects
- Shows that messages can flow between them

3. Messages

- Interaction between objects
- Represented as **labeled arrows along links**
- Numbered to indicate **sequence of messages**

Format:
`1: messageName(parameters)`

4. Sequence Numbers

- Indicate order of messages
- Example: `1`, `1.1`, `2`, `2.1`

5. Optional Elements

- Guard conditions `[ ]`
- Iteration loops `*`

Steps to Draw a Communication Diagram

1. Identify objects involved in the use case
2. Draw links between related objects
3. Label messages along the links
4. Number messages according to sequence
5. Add conditions and loops if necessary

Example: Login Process

Objects:

- `:User`
- `:LoginPage`
- `:AuthenticationServer`
- `:Database`

Messages:

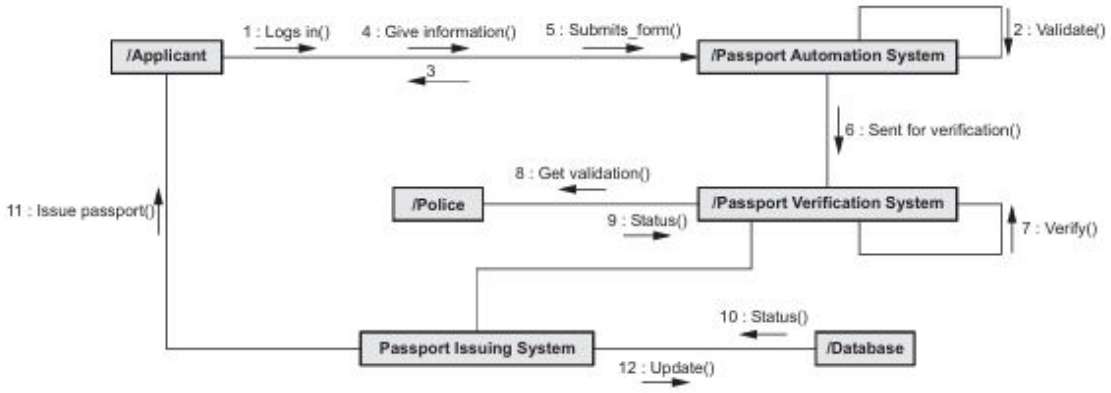
1. `:User → :LoginPage: enterCredentials()`
2. `:LoginPage → :AuthenticationServer: validateUser()`
3. `:AuthenticationServer → :Database: queryCredentials()`
4. `:AuthenticationServer → :LoginPage: validationResult()`
5. `:LoginPage → :User: showResult()`

Advantages

- Focuses on **object relationships**
- Shows both structure and dynamic behavior
- Helps identify collaboration between objects
- Useful for design and testing

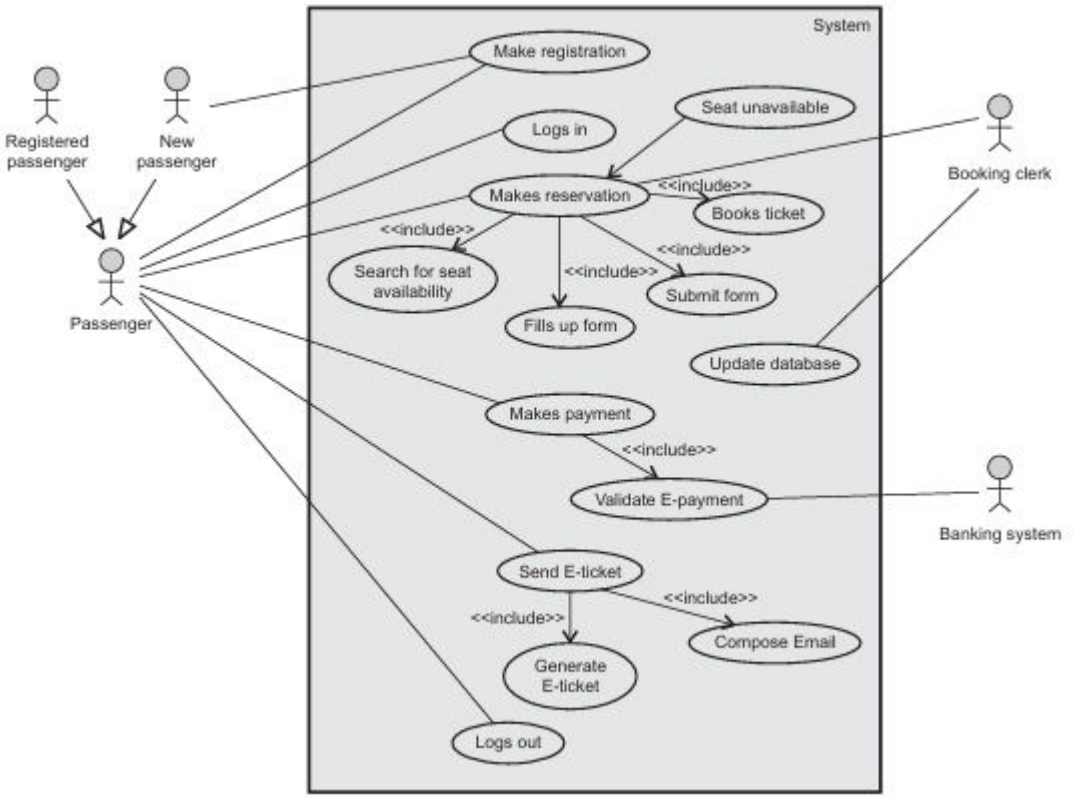
Limitations

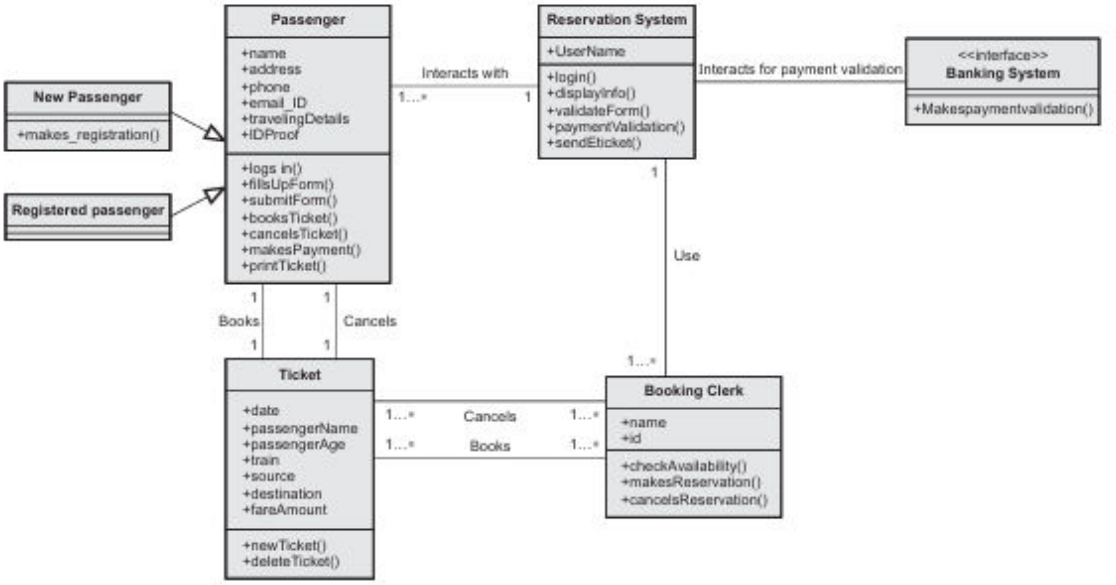
- Can become cluttered with many objects
- Harder to read than sequence diagrams for long interactions
- Time sequence is less explicit.



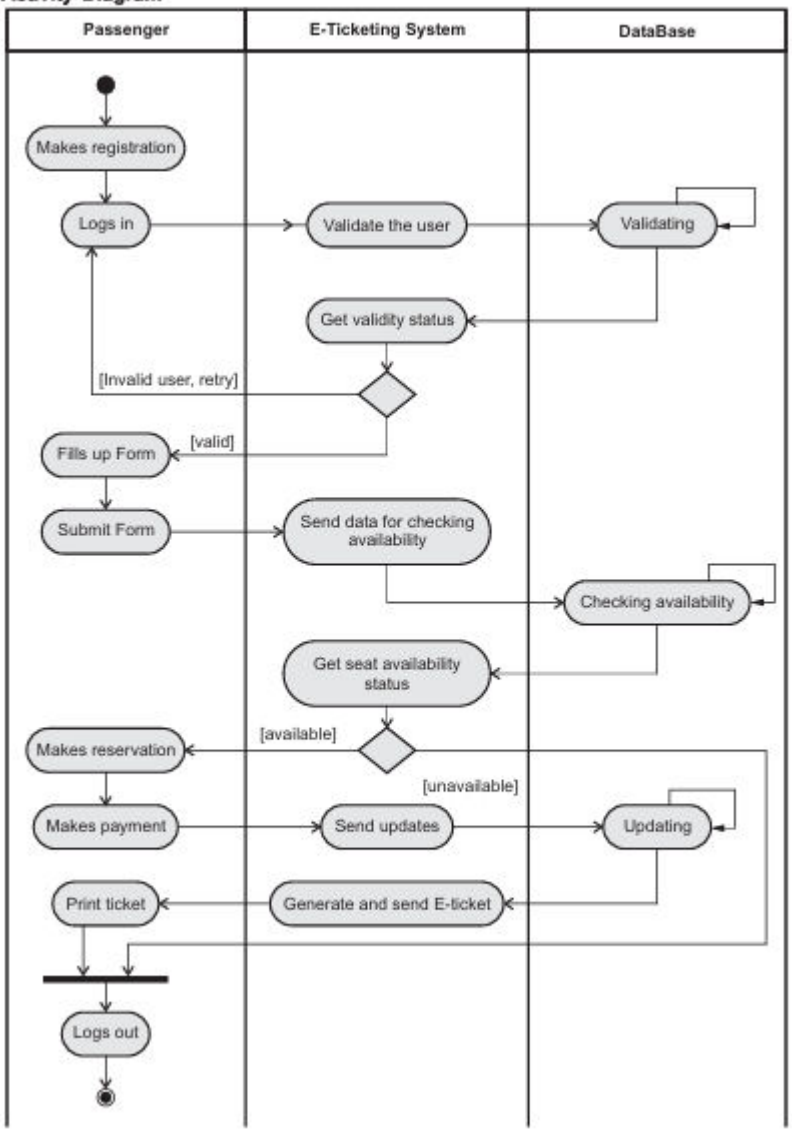
Design a UML diagram for E-Ticketing

Use Case Diagram

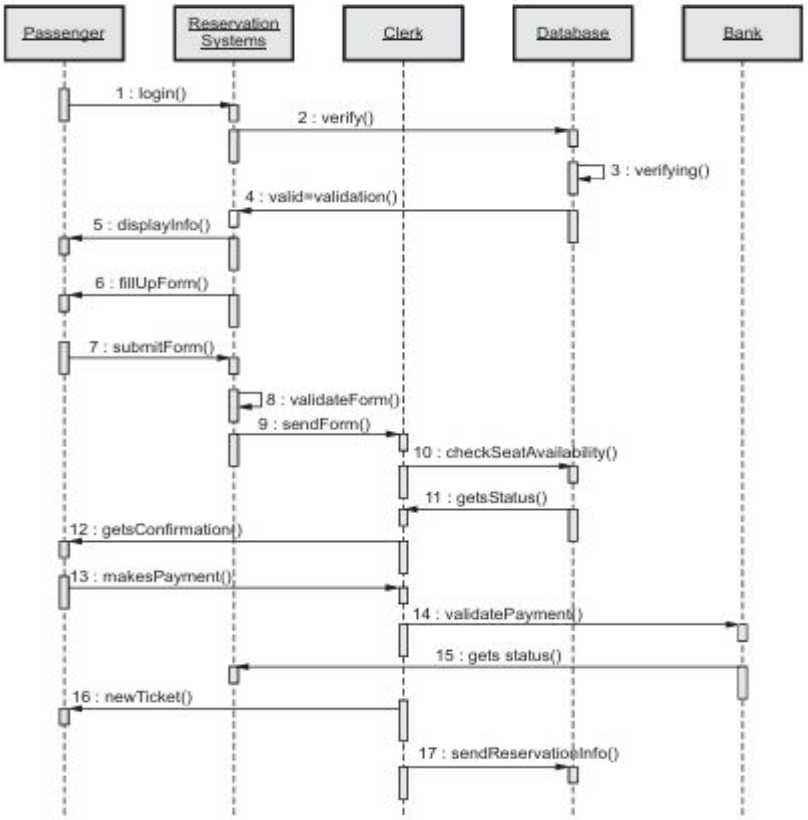




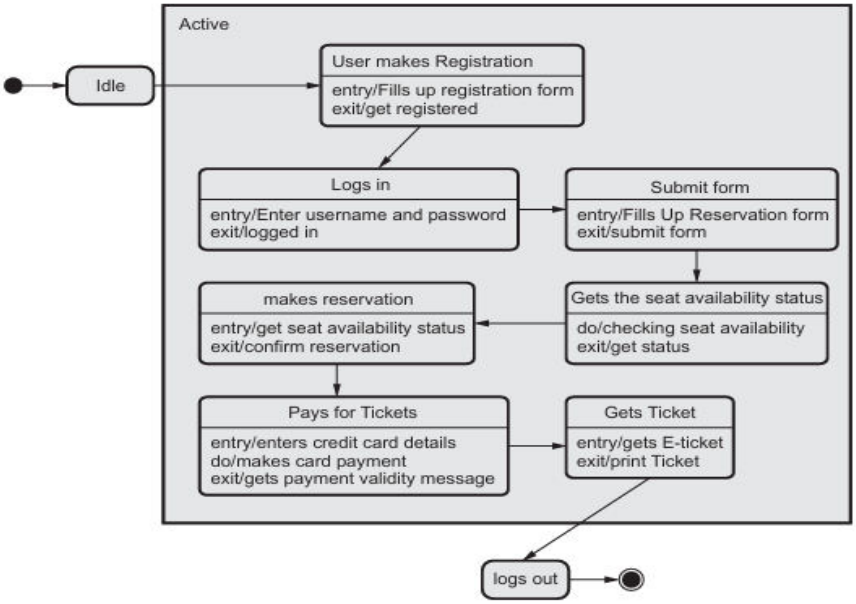
Activity Diagram



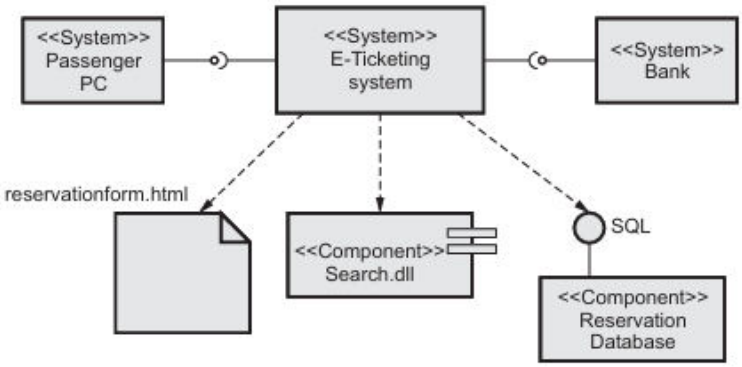
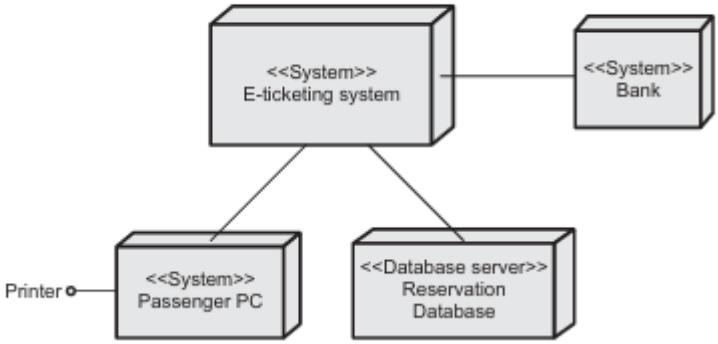
Sequence Diagram



State Chart Diagram



Component Diagram



Deployment Diagram